

24MCDAAT302

Design & Analysis of Algorithms

Module - 2

Greedy Strategy

Greedy Algorithm

- ❑ This method is used for solving optimization problems.
- ❑ An optimization problem is a problem that demands either maximum or minimum results.
- ❑ It is not an algorithm, but it is a technique.
- ❑ The main function of this approach is that the decision is taken on the basis of the currently available information. Whatever the current information is present, the decision is made without worrying about the effect of the current decision in future.
- ❑ This technique is basically used to determine the feasible solution that may or may not be optimal.
- ❑ The feasible solution is a subset that satisfies the given criteria.
- ❑ The optimal solution is the solution which is the best and the most favorable solution in the subset.

Characteristics of Greedy method

The following are the characteristics of a greedy method:

- ❑ To construct the solution in an optimal way, this algorithm creates two sets where one set contains all the chosen items, and another set contains the rejected items.
- ❑ A Greedy algorithm makes good local choices in the hope that the solution should be either feasible or optimal.

Components of Greedy Algorithm

The components that can be used in the greedy algorithm are:

- ❑ **Candidate set:** A solution that is created from the set is known as a candidate set.
- ❑ **Selection function:** This function is used to choose the candidate or subset which can be added in the solution.
- ❑ **Feasibility function:** A function that is used to determine whether the candidate or subset can be used to contribute to the solution or not.
- ❑ **Objective function:** A function is used to assign the value to the solution or the partial solution.
- ❑ **Solution function:** This function is used to intimate whether the complete function has been reached or not.

Optimization Problem :

Suppose there is a problem 'P'. I want to travel from A to B shown as below:

$P : A \rightarrow B$

The problem is that we have to travel this journey from A to B. There are various solutions to go from A to B. We can go from A to B by walk, car, bike, train, aero plane, etc. There is a constraint in the journey that we have to travel this journey within 12 hrs. If I go by train or aero plane then only, I can cover this distance within 12 hrs. There are many solutions to this problem but there are only two solutions that satisfy the constraint.

If we say that we have to cover the journey at the minimum cost. This means that we have to travel this distance as minimum as possible, so this problem is known as a minimization problem. Till now, we have two feasible solutions, i.e., one by train and another one by air. Since travelling by train will lead to the minimum cost so it is an optimal solution. An optimal solution is also the feasible solution, but providing the best result so that solution is the optimal solution with the minimum cost. There would be only one optimal solution.

The problem that requires either minimum or maximum result then that problem is known as an optimization problem. Greedy method is one of the strategies used for solving the optimization problems

Applications of Greedy Algorithm

- ❑ It is used in finding the shortest path.
- ❑ It is used to find the minimum spanning tree using the prim's algorithm or the Kruskal's algorithm.
- ❑ It is used in a job sequencing with a deadline.
- ❑ This algorithm is also used to solve the fractional knapsack problem

Disadvantages of using Greedy algorithm

Greedy algorithm makes decisions based on the information available at each phase without considering the broader problem. So, there might be a possibility that the greedy solution does not give the best solution for every problem.

Control Abstraction for Greedy algorithm

```
Algorithm Greedy(A : set; n : integer){
```

```
  MakeEmpty(solution);
```

```
  for(i = 2; i <= n; i++) {
```

```
    x = Select(A);
```

```
    if Feasible(solution, x) then
```

```
      solution = Union(solution, {x})
```

```
  }
```

```
  return solution
```

```
}
```

The function Greedy describes the essential way that a greedy algorithm will look, once a particular problem is chosen and the functions **Select**, **Feasible** and **Union** are properly implemented.

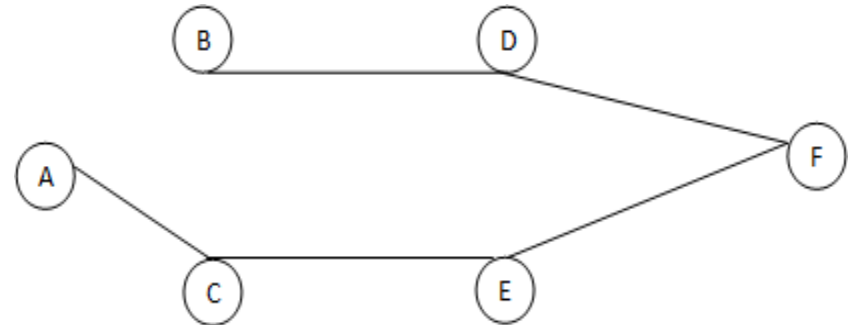
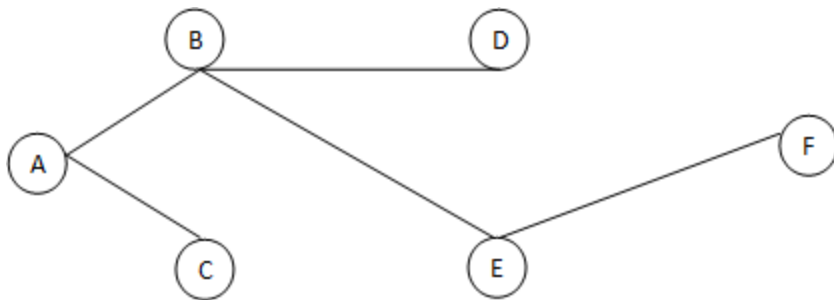
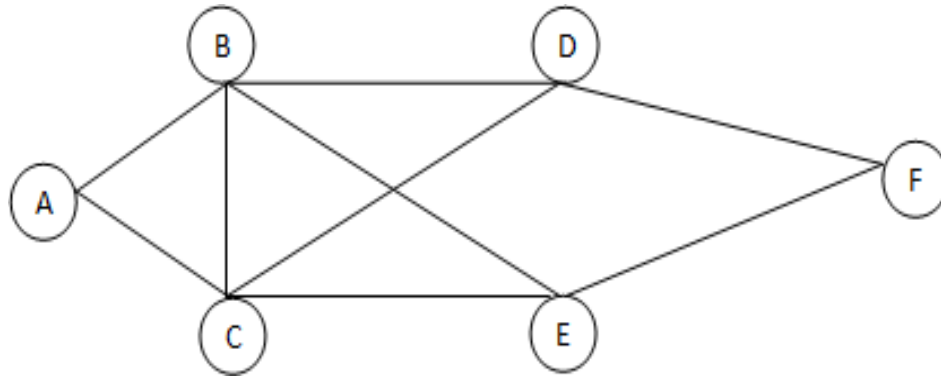
The function Select selects an input from A whose value is assign to x. Feasible is a Boolean valued function that determines if x can be included into the solution vector. The function Union combines x with the solution, and update the objective function.

BASIS FOR COMPARISON	GREEDY METHOD	DYNAMIC PROGRAMMING
Basic	Generates a single decision sequence.	Many decision sequence may be generated.
Reliability	Less reliable	Highly reliable
Follows	Top-down approach.	Bottom-up approach.
Solutions	Contain a particular set of feasible set of solutions.	There is no special set of feasible set of solution.
Efficiency	More	Less
Overlapping sub problems	Cannot be handled	Chooses the optimal solution to the sub problem.
Example	Fractional knapsack, shortest path.	0/1 Knapsack

Minimum Spanning Tree

The spanning tree is defined as a subgraph of an undirected graph in which all the vertices are connected.

Therefore any tree consisting of all the vertices of a graph is called a spanning tree.



The weight of a spanning tree for a weighted graph is the sum of the edge weights.

The Spanning tree with minimum total weight is called the minimum spanning tree (MST).

MST is a minimal subgraph of G' of a graph G , such that $V(G')=V(G)$ and $E(G') \subseteq E(G)$.

If there are N vertices in the graph, then the MST contains $(N-1)$ edges.

The MST has various applications including network design, approximation algorithms for NP-hard problems, cluster analysis etc.

There are two different algorithms to find out the MST

1. Kruskal's algorithm
2. Prim's algorithm

Kruskal's Algorithm

- ❑ This algorithm was developed by Joseph Kruskal in 1957.
- ❑ The Kruskal's algorithm creates the MST T by adding the edges with minimum cost one at a time to T .
- ❑ A minimum cost spanning tree T is built edge by edge without forming the cycle.
- ❑ We start with the edge of minimum cost.
- ❑ If there are several edges with the same cost, then we select any one of them and add it to the spanning tree T provided its addition does not form a cycle.

- ❑ We then add an edge with the next lowest cost and so on.
- ❑ We repeat this process until we have selected $N-1$ edges to form the complete MST.
- ❑ This algorithm selects the edges for addition in the MST in the increasing order of their cost.
- ❑ We have to remember here that the edges can only be added if they do not form a cycle.
- ❑ In another words we can arrange edges in the ascending order of their cost, and then decide which edge to add into the spanning tree if it does not form a cycle.

Algorithm Kruskal (G)

Input : The weighted connected undirected graph $G(V,E)$ with n vertices.

Output : Set of edges of minimum spanning tree.

Arrange the edges E in ascending order

$MST = \Phi$

Repeat until $|MST| + 1 < |V|$

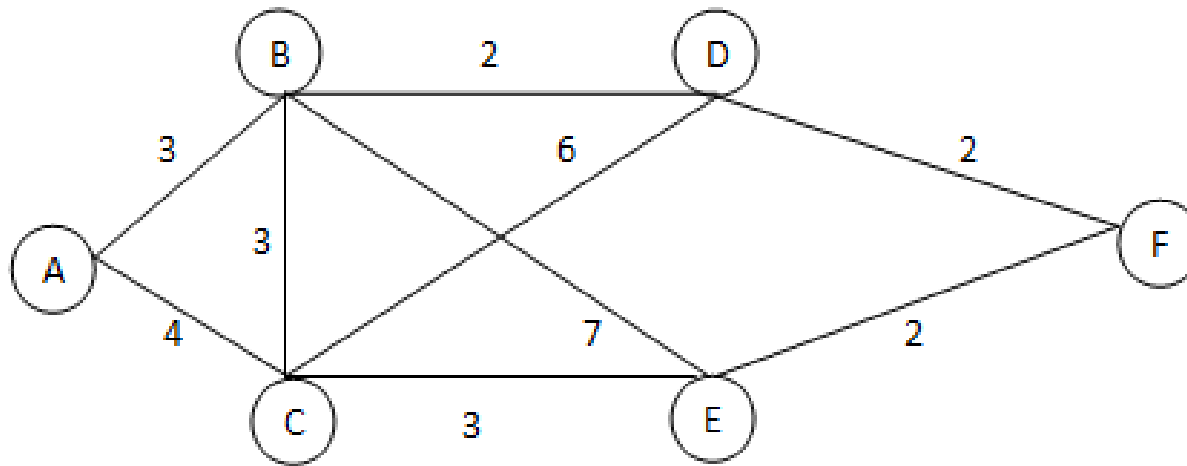
($n-1$ edges are added into MST)

Select the unused edge e with minimum weight

Add this edge to the MST if it does not form a cycle so

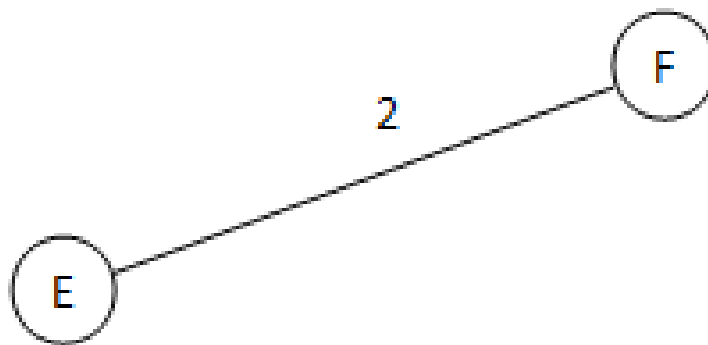
$MST = MST \cup \{e\}$

Return MST



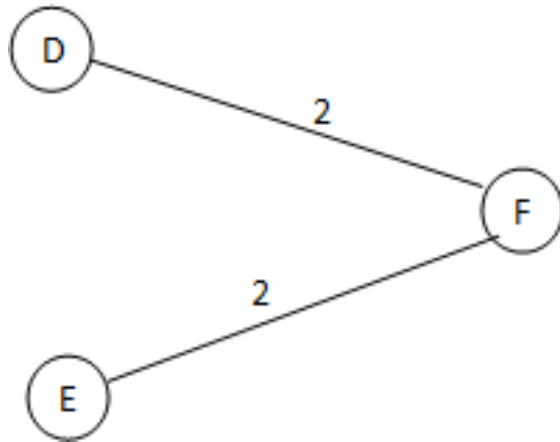
Step 1:

The edge with minimum cost are BD, DF and EF. Select any one



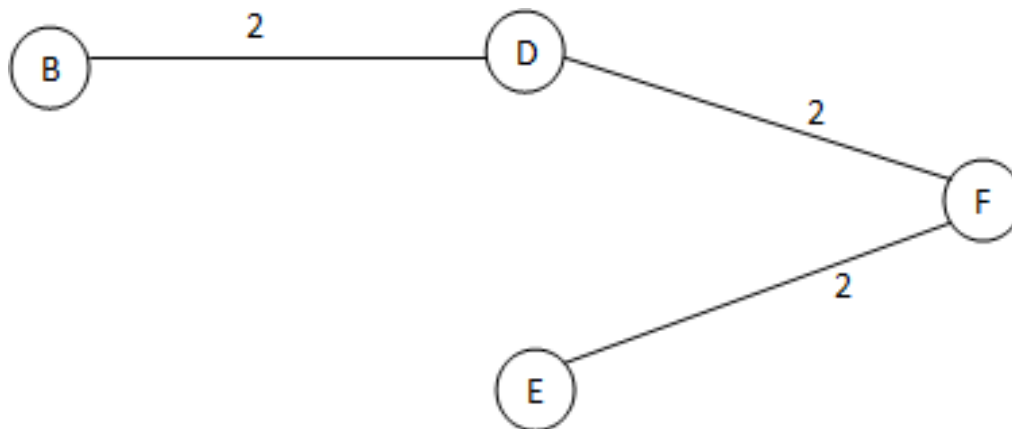
Step 2:

The edge with the next minimum cost are BD and DF. Select any one.



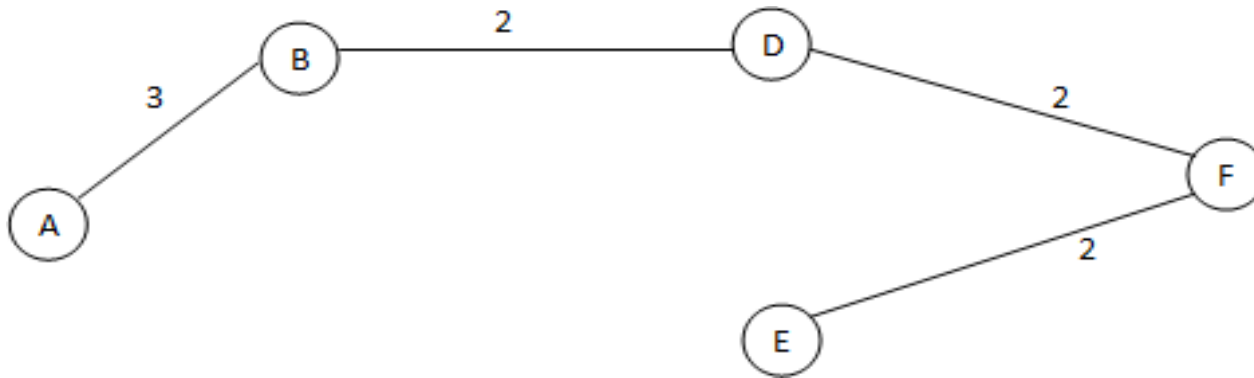
Step 3:

The edge with the next minimum cost is BD, so add it if it does not form a cycle.



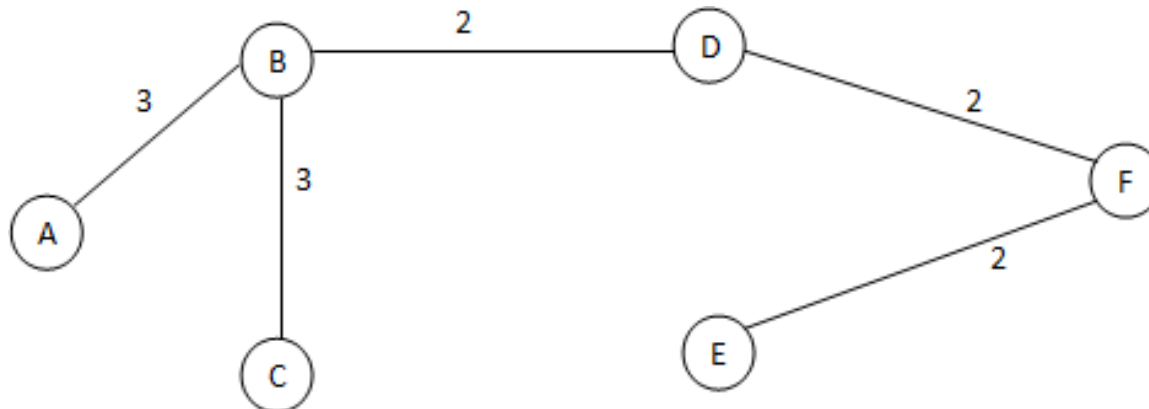
Step 4:

The edge with the next minimum cost are AB, BC and CE. So add any one that does not form a cycle.



Step 5:

The edge with the next minimum cost are BC and CE. Add any one that does not form a cycle.



Step 6:

Stop, since five edges($N-1$ where $N=6$) are included

Edge Cost Arrangement For Kruskal Method Of Finding MST

Edges	EF	DF	BD	AB	BC	CE	AC	CD	BE
Cost	2	2	2	3	3	3	4	6	7
Add	A	A	A	A	A				

☐ First row contains the edges in the ascending order of their cost

☐ Second row contains the cost

☐ Third row contains A if the corresponding edge is added

☐ The time complexity of Kruskal's algorithm is $O(n \log n)$.

Prim's Algorithm

- ❑ It is similar to Kruskal's algorithm in the manner that it also creates an MST T by adding the edges one at a time to T .
- ❑ Here, we start with the single vertex of graph(generally, the vertex from the lowest cost edge). Let it be A .
- ❑ Then, we find the cost of the edges incident on A , add the edge with minimum cost.
- ❑ Let it be AB , so we have two vertices in the set $V=\{A,B\}$.

- ❑ Again find the cost of the edges incident on A or B.
- ❑ Then add the edge with the lowest cost and repeat the same process until $N-1$ edges are added where N is the number of vertices in the graph.
- ❑ Prim's algorithm adds the edges with minimum cost and makes a locally optimal choice at each step.
- ❑ The edges can only be added if it does not form a cycle.

Comparison between Kruskal's and Prim's

❑ The tree is always connected when the MST is built using the Prim's method. The tree is not necessarily always connected when the MST is built using the Kruskal's method.

❑ Kruskal's Select the shortest edge in a network and Prim's Select any vertex

❑ In Prim's method, the edge with the minimum weight is selected from the vertex already in the MST, but in Kruskal's method, the edge selected with the minimum cost need not necessarily be in MST.

❑ Both the methods avoid formation of cycle when adding the edge

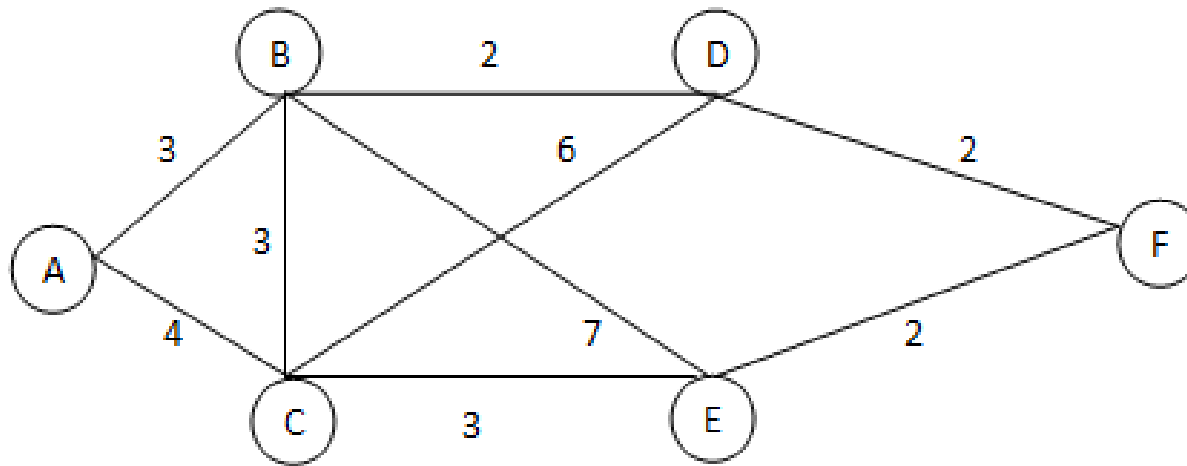
Algorithm Prim (G)

Input : The weighted connected undirected graph $G(V,E)$ with n vertices.

Output : Set of edges (MST) Of minimum spanning tree.

1. Set $R=s$; the starting vertex (Any vertex may be chosen as starting vertex)
2. Set $T=V-s$
3. $MST=\Phi$
4. Set $Noofedgesincluded=0$
5. Repeat until $Noofedgesincluded + 1 < n$
($n-1$ edges are added into MST)
6. Select an unused edge e of minimum weight incident to any vertex u in R , that is,
 $e=\{u,v\}$ where $u \in R$ and $v \in T$
 - a. $MST=MST \cup \{e\}$
(add this edge to the MST if it does not form a cycle)
 - b. Set $R= R \cup v$, $T=T-u$
 - c. $Noofedgesincluded= Noofedgesincluded + 1$
7. Return MST

The time complexity of Prim's algorithm is $O(n^2)$



Step 1:

Select the vertex A



$MST = \Phi$

$R = \{A\}$

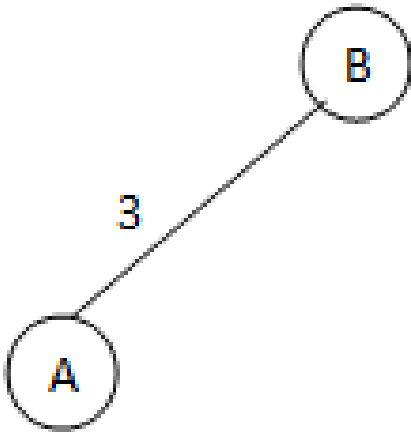
$T = \{B, C, D, E, F\}$

Step 2:

The edges incident on A are AB and AC. So add the edge with the minimum cost.

Since cost of AB is 3 and cost of AC is 4, add AB.

$$\text{Min}\{AB, AC\} = \text{Min}\{3, 4\} = 3 = AB$$



MST = {AB}

R = {A, B}

T = {C, D, E, F}

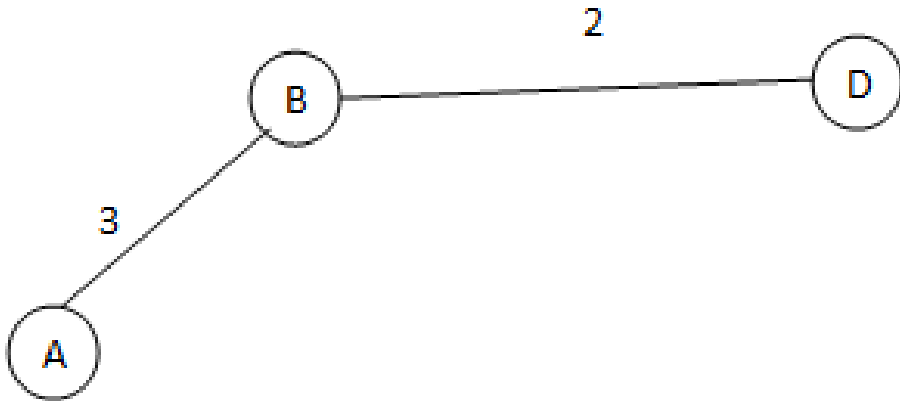
No of edges included = 1

Step 3:

Find the edge incident on either A or B with minimum cost. AB and AC are the edges incident on A and BC, BD and BE are incident on B. Since AB is already used (included in MST), ignore it.

$$\text{Min}\{AC, BD, BE, BC\} = \text{Min}\{4, 2, 7, 3\} = 2 = BD$$

Therefore, add the edge BD.



MST = {AB, BD}

R = {A, B, D}

T = {C, E, F}

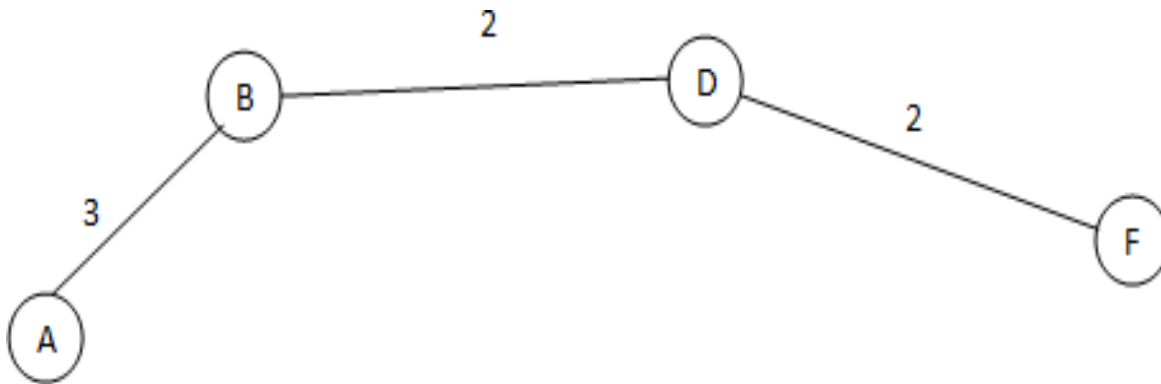
No of edges included = 2

Step 4:

Find the edge incident on A,B or D. AB and AC are the edges incident on A; BC, BD and BE are incident on B and BD, CD and DF are incident on D. Since AB and BD are already used(included). Ignore them.

$$\text{Min}\{AC, BE, BC, CD, DF\} = \text{Min}\{4, 7, 3, 6, 2\} = 2 = DF$$

Therefore, add the edge DF.



$MST = \{AB, BD, DF\}$

$R = \{A, B, D, F\}$

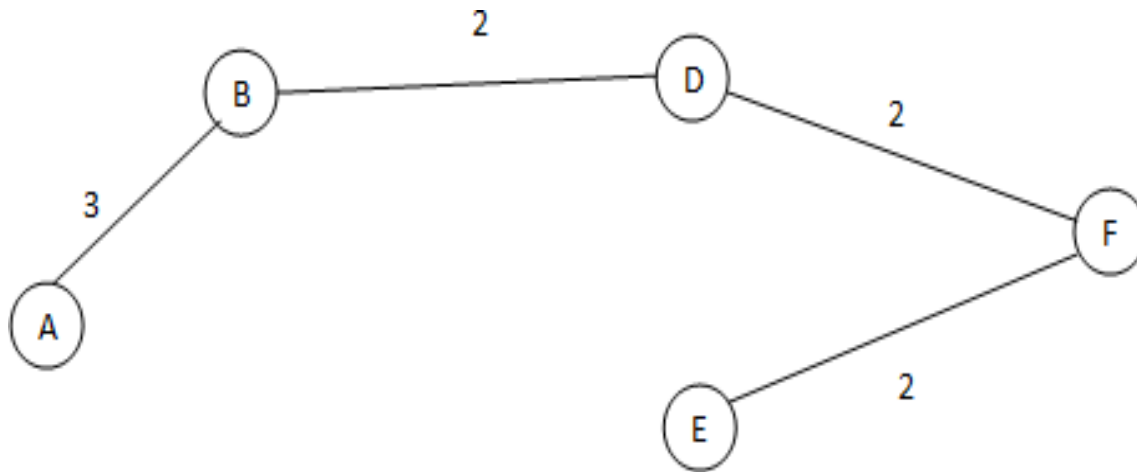
$T = \{C, E\}$

No of edges included = 3

Step 5:

Find the edge incident on A,B,D or F. The edges incident on these vertices and which are not included in the tree are AC, BC, BE, DC, EF. The edge with minimum cost among all these edges is EF. Therefore, add this edge to MST.

$$\text{Min}\{AC, BC, BE, DC, EF\} = \text{Min}\{4, 3, 7, 6, 2\} = 2 = EF$$



MST = {AB, BD, DF, EF}

R = {A, B, D, F, E}

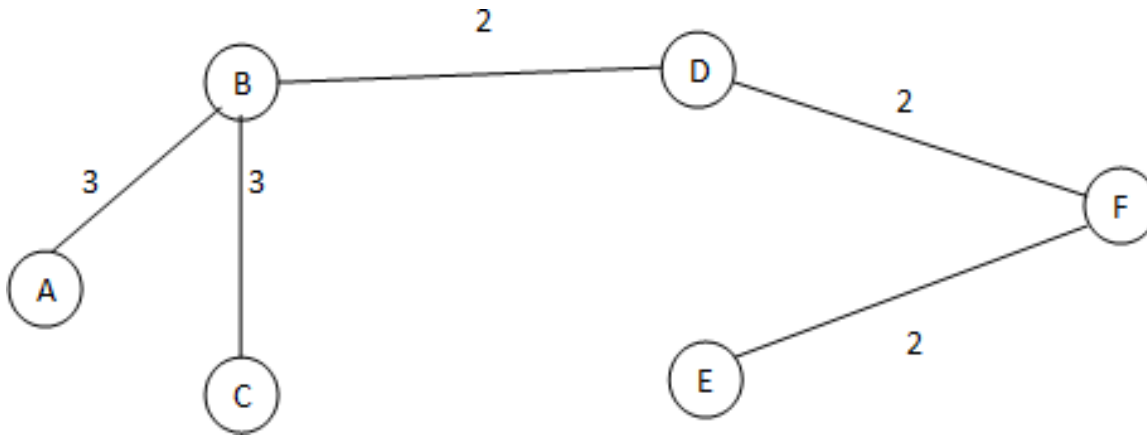
T = {C}

No of edges included = 4

Step 6:

Find the edge incident on A,B,D,F or E. The edges incident on these vertices and which are not included in the tree are AC, BC, BE, DC, EC. The edge with minimum cost among all these edges are BC and EC. Therefore, add either BC or EC to MST.

$$\text{Min}\{AC, BC, BE, DC, EC\} = \text{Min}\{4, 3, 7, 6, 3\} = 3 = \{BC, EC\}$$



$MST = \{AB, BD, DF, EF, BC\}$

$R = \{A, B, D, F, E, C\}$

$T = \Phi$

No of edges included = 5

Step 7:

Stop, since five edges ($N-1$ where $N=6$) are included and R contains all the vertices

Knapsack Problem



10 oz., \$1,000



Max Weight: 400 oz.



100 oz., \$2,000



300 oz., \$4,000



1 oz., \$5,000



200 oz., \$5,000

Combinatorial optimization is an emerging field at the forefront of combinatorics and theoretical computer science that aims to use combinatorial techniques to solve discrete optimization problems. A discrete optimization problem seeks to determine the best possible solution from a finite set of possibilities.

From a computer science perspective, combinatorial optimization seeks to improve an algorithm by using mathematical methods either to reduce the size of the set of possible solutions or to make the search itself faster.

Combinatorial optimization is a topic that consists of finding an optimal object from a finite set of objects. In many such problems, exhaustive search is not tractable. It operates on the domain of those optimization problems in which the set of feasible solutions is discrete or can be reduced to discrete, and in which the goal is to find the best solution. Some common problems involving combinatorial optimization are the travelling salesman problem ("TSP"), the minimum spanning tree problem ("MST"), and the knapsack problem.

Applications for combinatorial optimization include, but are not limited to:

- ❑ Developing the best airline network of spokes and destinations
- ❑ Deciding which taxis in a fleet to route to pick up fares
- ❑ Determining the optimal way to deliver packages
- ❑ Working out the best allocation of jobs to people

Knapsack Problem

Based On The Concept Of Maximizing the benefit

A thief robbing a general store has a knapsack of capacity "**W**", then he would try to pack the knapsack with only those items that maximize the total cost of selected items.

The thief can carry a maximum weight of "**W**" in the knapsack, so there are three options:

- ☐ Collect the most costly items in order.
- ☐ Collect the light weighted items in order.
- ☐ Collect the precious (ratio of value/weight) items in order

There are three greedy choices:

1. Arrange the items in descending order of their value and then pick them in order.
2. Arrange the items in ascending order of their weights or size and then pick them in order.
3. Arrange the items in descending order of the ratio(value/weight) and pick them in order

The above mentioned strategy is generalized to the selection of objects from a set of "**n**" objects for the knapsack of capacity "**W**".

Each object "**i**" has a positive weight " **w_i** " and value " **v_i** ".

The objects are selected in order to maximize the total value of the selected object in the knapsack without breaking the knapsack.

Example :

knapsack size : 60kg

Objects for selection : 3

Object	A	B	C
Weight (w)	40	10	20
Value (v)	280	100	120

1 Arrange the items in descending order of their value and then pick them in order without breaking the knapsack

Object	A	C	B
Value (v)	280	120	100
Weight (w)	40	20	10

Knapsack size $W=60$

- ❑ Select the most valuable item A with value 280 and weight 40, so now we can only keep $W-w_i$ that is , $60-40=20$ into the knapsack.
- ❑ Select the next valuable item C with value 120 and weight 20, so we can now keep $20-20=0$ into the knapsack.
- ❑ We can't select B because the knapsack is full
- ❑ The total value of selected items is $280+120=400$

2 Arrange the items in ascending order of their weight and then pick them in order without breaking the knapsack.

Object	B	C	A
Weight (w)	10	20	40
Value (v)	100	120	280

Knapsack size $W=60$

Select the most light item B with weight 10 and value 100, so now we can only keep $60-10=50$ into the knapsack

Select the next light weight item C with weight 20 and value 120 , so now we can keep $50-20=30$ into knapsack.

We can't select the next item A because weight of A $40 > 30$

The total value of the selected items is $100+120=220$.

3 Arrange the items in descending order of their ratio(value/weight) and then pick them in order without breaking the knapsack.

Object	B	A	C
Value (v)	100	280	120
Weight (w)	10	40	20
Profit ratio	10	7	6

Knapsack size $W=60$

select the most valuable item B with value 100 and weight 10, so now we can only keep $W-w_i$ that is , $60-10=50$ into the knapsack.

Select the next valuable item A with value 280 and weight 40, so we can now keep $50-40=10$ into the knapsack.

Now we can't select C because its weight (>10) will break the knapsack.

The total value of selected items is $100+280=380$

Applying all these strategies to 0/1 knapsack problem does not give the optimum solution, therefore greedy algorithms do not work for the 0/1 knapsack problem.

But greedy algorithms works well for fractional version of the problem where fraction of the item may also be included into the knapsack.

$$\sum_{i=1}^n x_i w_i \leq W \text{ and } \sum_{i=1}^n x_i v_i \text{ is maximized}$$

The purpose of inclusion of items will remain the same to maximize the total profit.

1 Arrange the items in descending order of their value and then pick them in order without breaking the knapsack

Object	A	C	B
Value (v)	280	120	100
Weight (w)	40	20	10

Knapsack size $W=60$

❑ Select the most valuable item A with value 280 and weight 40, so now we can only keep $W-w_i$ that is , $60-40=20$ into the knapsack.

❑ Select the next valuable item C with value 120 and weight 20, so we can now keep $20-20=0$ into the knapsack.

The total will be 400.

2. Arrange the items in ascending order of their weight and then pick them in order without breaking the knapsack.

Object	B	C	A
Weight (w)	10	20	40
Value (v)	100	120	280

Knapsack size $W=60$

Select the most light item B with weight 10 and value 100, so now we can only keep $60-10=50$ into the knapsack

Select the next light weight item C with weight 20 and value 120 , so now we can keep $50-20=30$ into knapsack.

We can't select the next item A because weight of A $40 > 30$

Select the fraction of A then it will be like $(30/40)*280 = 210$

The total value will be $100+120+210=430$

3 Arrange the items in descending order of their ratio(value/weight) and then pick them in order without breaking the knapsack.

Object	B	A	C
Value (v)	100	280	120
Weight (w)	10	40	20
Profit ratio	10	7	6

Knapsack size $W=60$

Select the most valuable item B with value 100 and weight 10, so now we can only keep $W-w_i$ that is , $60-10=50$ into the knapsack.

Select the next valuable item A with value 280 and weight 40, so we can now keep $50-40=10$ into the knapsack.

Now we can't select C because its weight (>10) will break the knapsack.

Take the fraction of C ie, $(10/20)*120=60$

Total value will be like $100+280+60=440$

The fractional knapsack problem is solved by the greedy technique using the ratio of the total value to the total weight as the greedy choice for selection of objects to gain the maximum profit.

The most valuable item is selected first, then the next valuable item and eventually the knapsack is filled by taking as much of the last item as will fit.

Greedy Method to solve the fractional knapsack problem

1. Compute the profit value ratio of each item x_i .
2. Arrange each item in descending order of their profit value ratio.
3. Select an item in order from the sorted list provided the selected item does not break the knapsack.

Algorithm FractionalKnapsack(S, W)

Input: Set $S = \{i_1, i_2, i_3, \dots, i_n\}$ of items. Each item is assigned with the value v_i and weight w_i . The maximum weight a knapsack can handle is W

Output: Amount x_i of each item i to maximize benefit with weight at most W .

Repeat for each item $i=1$ to n

 set $x_i \leftarrow 0$ (selection part of item i is set to zero for every item initially)

 set $p_i \leftarrow v_i / w_i$ {profit value of each item}

End repeat

 set $cw \leftarrow 0$ and $i \leftarrow 1$ {set current total weight to 0 and start index to 1}

Repeat while $cw < W$ and $i \leq n$

 remove item i with highest profit value p_i from set S

 if $(cw + w_i) \leq W$ then

 set $x_i \leftarrow 1$ and $cw \leftarrow cw + w_i$

 else

 set $x_i \leftarrow \{W - cw\} / w_i$

$cw = W$

 endif

 set $i \leftarrow i + 1$

endwhile

return(x)

Job sequencing with deadlines

Job Sequencing with deadlines Greedy Algorithm

We adopt a greedy algorithm to determine how the next job is selected for an optimal solution. The greedy algorithm described below always gives an optimal solution to the job sequencing problem.

Step-01:

Sort all the given jobs in the decreasing order of their profits.

Step-02:

Check the value of maximum deadline. Then, draw a Gantt chart where maximum time on Gantt chart is the value of maximum deadline.

Step-03:

Pick up the jobs one by one and then put them on the Gantt chart as far as possible from 0 (starting point) such that the job gets completed before its deadline.

Given the following jobs, their deadlines and associated profits as shown-

Jobs	J1	J2	J3	J4	J5	J6
Deadlines	5	3	3	2	4	2
Profits	200	180	190	300	120	100

Answer the following questions-

1. Write the optimal schedule that gives maximum profit.
2. Are all the jobs completed in the optimal schedule?
3. What is the maximum earned profit?

Solution-

Step-01:

Sort all the given jobs in the decreasing order of their profits-

Jobs	J4	J1	J3	J2	J5	J6
Deadlines	2	5	3	3	4	2
Profits	300	200	190	180	120	100

Step-02:

Value of maximum deadline = 5

So, draw a Gantt chart with maximum time on Gantt chart = 5 units as shown-

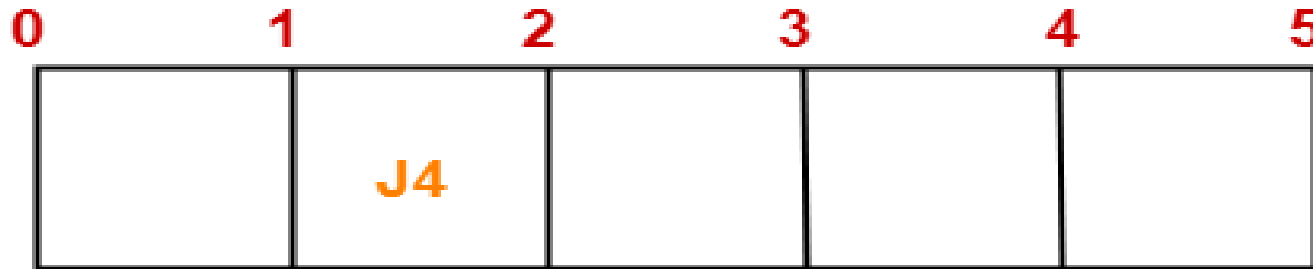


Gantt Chart

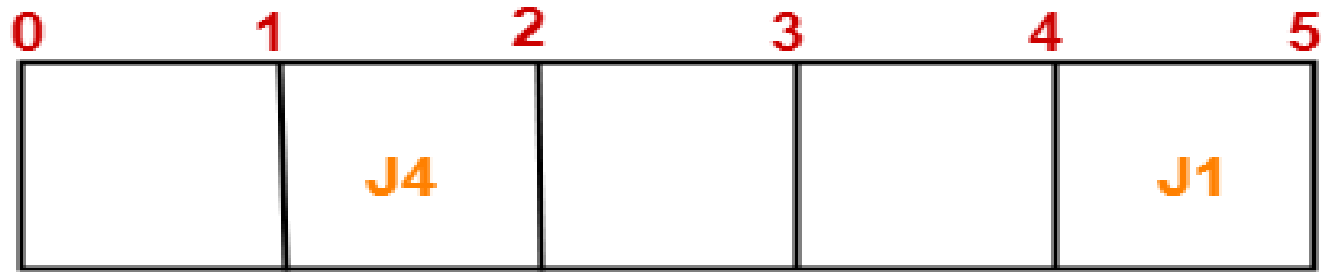
Step-03:

Now, we will take each job one by one in the order they appear in step-01 and place them on Gantt chart as far as possible from 0 (starting point)

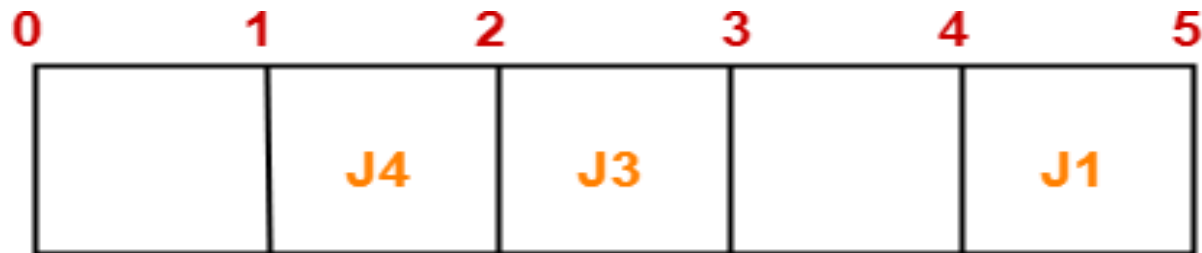
- First, we take job J4.
- Since, its deadline is 2, so we place it in the first empty cell before deadline 2 as-



- Now, we take job J1.
- Since, its deadline is 5, so we place it in the first empty cell before deadline 5 as-



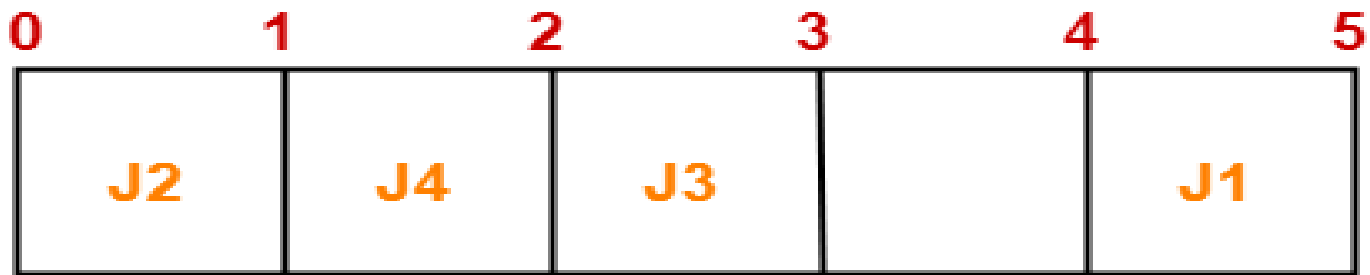
- Now, we take job J3.
- Since, its deadline is 3, so we place it in the first empty cell before deadline 3 as-



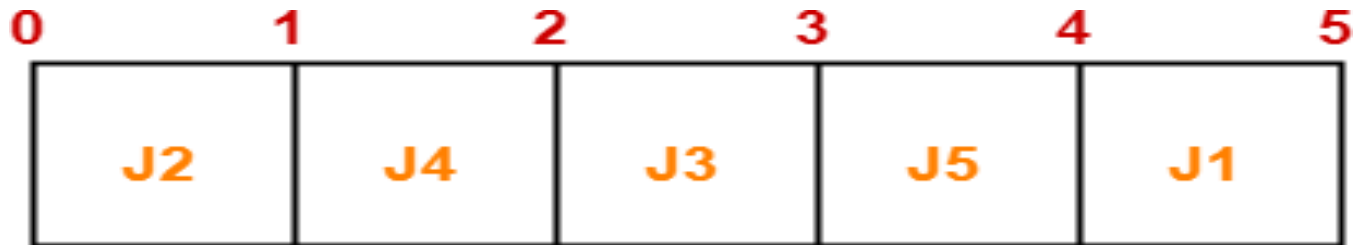
Now, we take job J2.

Since, its deadline is 3, so we place it in the first empty cell before deadline 3.

Since, the second and third cells are already filled, so we place job J2 in the first cell as-



- Now, we take job J5.
- Since, its deadline is 4, so we place it in the first empty cell before deadline 4 as



Now,

The only job left is job J6 whose deadline is 2.

All the slots before deadline 2 are already occupied.

Thus, job J6 can not be completed.

The optimal schedule is-

J2 , J4 , J3 , J5 , J1

This is the required order in which the jobs must be completed in order to obtain the maximum profit.

All the jobs are not completed in the optimal schedule.

This is because job J6 could not be completed within its deadline.

Maximum earned profit

= Sum of profits of all jobs in optimal schedule

= Profit of job J2 + Profit of job J4 + Profit of job J3 + Profit of job J5 + Profit of job J1

= 180 + 300 + 190 + 120 + 200

= 990 units

JobSequencing (S)

Input : The set S of jobs each with deadline d_i and profit p_i . These jobs are arranged in descending order of their profit value and the jobs are renamed with numerical as 1,2,3,....., N, that is, $S=\{1,2,3,..,N\}$.

Output : The set A with maximum number of jobs.

1. Set $N \leftarrow \text{length}[S]$
2. Set $A \leftarrow 1$ (activity with the most profit value)
3. Repeat for $i \leftarrow 1$ to N (Repeat until all the jobs are considered)
 - a. Assign the job to the slot $[m-1, m]$ if slot $[m-1, m]$ is free. Here m is $1 \leq m \leq d_i$
 - b. Set $A \leftarrow A \cup \{i\}$

Return A

**Dynamic Programming:
Control Abstraction, Principle of
Optimal Substructure**

Dynamic Programming

Dynamic Programming

- ❑ Dynamic programming is a technique that breaks the problems into sub-problems, and saves the result for future purposes so that we do not need to compute the result again.
- ❑ The sub problems are optimized to optimize the overall solution is known as optimal substructure property.
- ❑ The main use of dynamic programming is to solve optimization problems.
- ❑ The dynamic programming guarantees to find the optimal solution of a problem if the solution exists.
- ❑ The definition of dynamic programming says that it is a technique for solving a complex problem by first breaking into a collection of simpler sub problems, solving each sub problem just once, and then storing their solutions to avoid repetitive computations.

How does the dynamic programming approach work?

- ❑ It breaks down the complex problem into simpler sub problems.
- ❑ It finds the optimal solution to these sub-problems.
- ❑ It stores the results of sub problems .The process of storing the results of sub problems is known as Memoization.
- ❑ It reuses them so that same sub-problem is calculated more than once.
- ❑ Finally, calculate the result of the complex problem.

Dynamic Programming

- ❑ Dynamic programming is a technique that breaks the problems into sub-problems, and saves the result for future purposes so that we do not need to compute the result again.
- ❑ The sub problems are optimized to optimize the overall solution is known as optimal substructure property.
- ❑ The main use of dynamic programming is to solve optimization problems. Here, optimization problems mean that when we are trying to find out the minimum or the maximum solution of a problem.
- ❑ The dynamic programming guarantees to find the optimal solution of a problem if the solution exists

The following are the steps that the dynamic programming follows:

- ❑ It breaks down the complex problem into simpler subproblems.
- ❑ It finds the optimal solution to these sub-problems.
- ❑ It stores the results of subproblems (memoization). The process of storing the results of subproblems is known as memorization.
- ❑ It reuses them so that same sub-problem is calculated more than once.
- ❑ Finally, calculate the result of the complex problem.

Control Abstraction

- ❑ Dynamic programming (DP) splits the large problem at every possible point. When the problem becomes sufficiently small, DP solves it.
- ❑ Dynamic programming is bottom up approach, it finds the solution of the smallest problem and constructs the solution of the larger problem from already solved smaller problems.
- ❑ To avoid recomputation of the same problem, DP saves the result of sub problems into the table. When next time same problem encounters, the answer is retrieved from the table by lookup procedure.

Control abstraction for dynamic programming is shown below :

Algorithm DYNAMIC_PROGRAMMING (P)

if solved(P) **then**

 return lookup(P)

else

 Ans $\leftarrow$ SOLVE(P)

 store (P, Ans)

end

Function SOLVE(P)

if sufficiently small(P) **then**

 solution(P) // Find solution for sufficiently small problem

else

 Divide P into smaller sub problems P1, P2, ..., Pn

 Ans1 $\leftarrow$ DYNAMIC_PROGRAMMIN(P1)

 Ans2 $\leftarrow$ DYNAMIC_PROGRAMMIN(P2)

 ...

 Ansn $\leftarrow$ DYNAMIC_PROGRAMMIN(Pn)

return(combine(Ans1, Ans2, ... , Ansn))

end

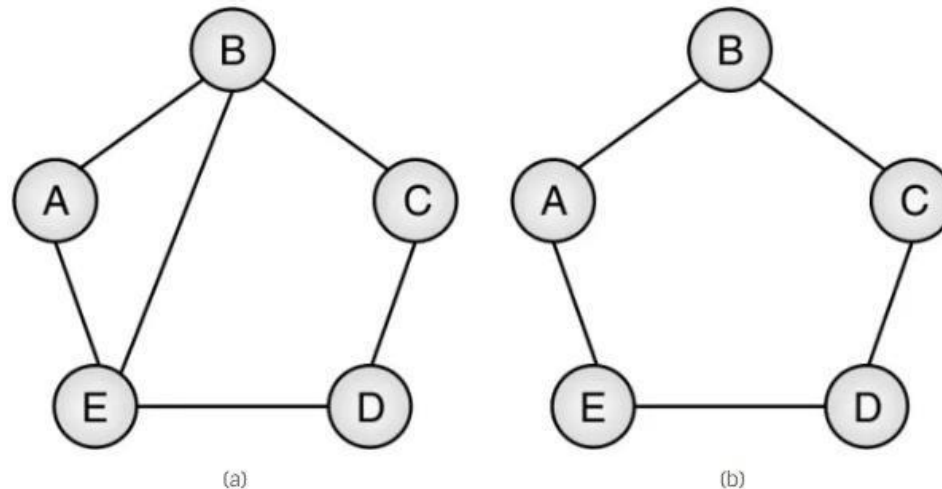
Principle of Optimality

□ The principle of optimality is the heart of dynamic programming. It states that to find the optimal solution of the original problem, a solution of each sub problem also must be optimal. It is not possible to derive optimal solution using dynamic programming if the problem does not possess the principle of optimality.

□ Shortest path problem satisfies the principle of optimality. If $A - X_1 - X_2 - \dots - X_n - B$ is the shortest path between nodes A and B, then any sub path X_i to X_j must be shortest. If there exist multiple paths from X_i to X_j , and the selected path is not minimum, then obviously the path from A to B cannot be shortest.

❑ For example : In Figure (a) shortest path from A to C is $A - B - C$. There exist two paths from B to C. One is $B - C$ and other is $B - E - D - C$. But $B - C$ is the shortest path so we should add that one in final solution.

❑ On the other hand, longest path problem does not satisfy the principle of optimality. In Figure (b), the longest non-cyclic path from A to D is $A - B - C - D$. In this path, sub path from B to C is just an edge joining B and C. However, BC itself is not the longest path, because longest non-cyclic path from B to C is $B - A - E - D - C$. Thus the sub path of longest path may not be longest always, so violates principle of optimality.



Optimal Substructure:

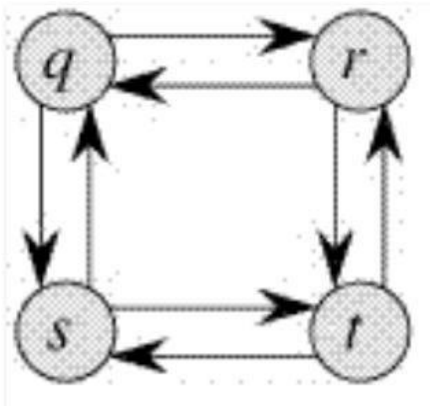
A given problem has Optimal Substructure Property if optimal solution of the given problem can be obtained by using optimal solutions of its subproblems.

For example, the Shortest Path problem has following optimal substructure property:

If a node x lies in the shortest path from a source node u to destination node v then the shortest path from u to v is combination of shortest path from u to x and shortest path from x to v . The standard All Pair Shortest Path algorithm like Floyd–Warshall and Single Source Shortest path algorithm for negative weight edges like Bellman–Ford are typical examples of Dynamic Programming.

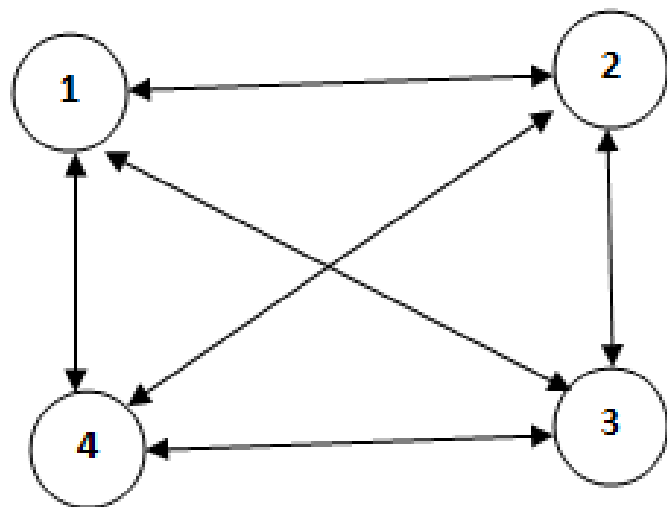
On the other hand, the Longest Path problem doesn't have the Optimal Substructure property. Here by Longest Path we mean longest simple path (path without cycle) between two nodes.

Consider the following unweighted graph



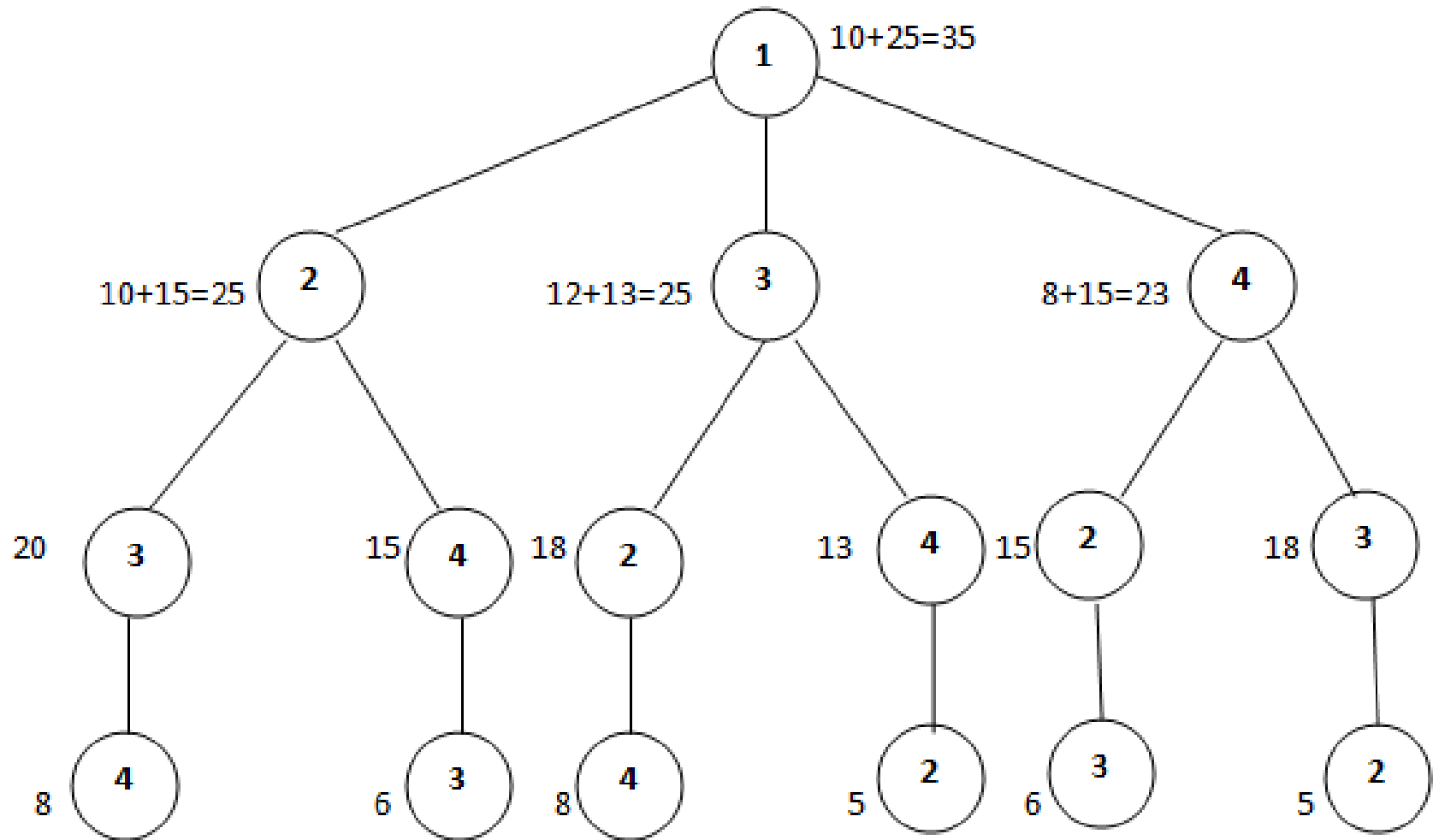
There are two longest paths from q to t : $q \rightarrow r \rightarrow t$ and $q \rightarrow s \rightarrow t$. Unlike shortest paths, these longest paths do not have the optimal substructure property. For example, the longest path $q \rightarrow r \rightarrow t$ is not a combination of longest path from q to r and longest path from r to t , because the longest path from q to r is $q \rightarrow s \rightarrow t \rightarrow r$ and the longest path from r to t is $r \rightarrow q \rightarrow s \rightarrow t$.

Travelling Sales Person Problem



	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Brute Force Approach



Solving it from bottom-up

Cost : 4th vertex to vertex 1 = 8 (From Matrix)

Cost : 3rd vertex to vertex 1 = 6 (From Matrix)

Cost : 2nd vertex to vertex 1 = 5 (From Matrix)

Starting from 4 to 1 if the person reached vertex 3 then the cost will be
 $3 \rightarrow 4 + 4 \rightarrow 1 = 12 + 8 = 20$

Starting from 3 to 1 if the person reached vertex 4 then the cost will be
 $4 \rightarrow 3 + 3 \rightarrow 1 = 9 + 6 = 15$

The cost of vertex 2 (ie, the subtree starts with root node '2')

Starting from vertex 4 if the person reached vertex 2 then the cost will be
 $2 \rightarrow 3 + 3 \rightarrow 4 + 4 \rightarrow 1 = 9 + 20 = 29$

Starting from vertex 3 if the person reached vertex 2 then the cost will be
 $2 \rightarrow 4 + 3 \rightarrow 4 + 4 \rightarrow 1 = 10 + 15 = 25$

Since we need the low cost we will select the value 25

Starting from 4 to 1 if the person reached vertex 2 then the cost will be
 $2 \rightarrow 4 + 4 \rightarrow 1 = 10 + 8 = 18$

Starting from 2 to 1 if the person reached vertex 4 then the cost will be
 $4 \rightarrow 2 + 2 \rightarrow 1 = 8 + 5 = 13$

The cost of vertex 3 (ie, the subtree starts with root node '3')
Starting from vertex 4 if the person reached vertex 3 then the cost will be
 $3 \rightarrow 2 + 2 \rightarrow 4 + 4 \rightarrow 1 = 13 + 18 = 31$

Starting from vertex 2 if the person reached vertex 3 then the cost will be
 $3 \rightarrow 4 + 4 \rightarrow 2 + 2 \rightarrow 1 = 12 + 13 = 25$

Since we need the low cost we will select the value 25

Starting from 3 to 1 if the person reached vertex 4 then the cost will be
 $2 \rightarrow 3 + 3 \rightarrow 1 = 9 + 6 = 15$

Starting from 2 to 1 if the person reached vertex 4 then the cost will be
 $3 \rightarrow 2 + 2 \rightarrow 1 = 13 + 5 = 18$

The cost of vertex 4 (ie, the subtree starts with root node '4')
Starting from vertex 3 if the person reached vertex 4 then the cost will be
 $4 \rightarrow 2 + 2 \rightarrow 3 + 3 \rightarrow 1 = 8 + 15 = 23$

Starting from vertex 2 if the person reached vertex 4 then the cost will be
 $4 \rightarrow 3 + 3 \rightarrow 2 + 2 \rightarrow 1 = 9 + 18 = 27$

Since we need the low cost we will select the value 23

Cost of the vertex 1 calculated through vertex 2 = $25 + 10(1 \rightarrow 2) = 35$

Cost of the vertex 1 calculated through vertex 3 = $25 + 15(1 \rightarrow 3) = 40$

Cost of the vertex 1 calculated through vertex 4 = $23 + 20(1 \rightarrow 4) = 43$

Cost of the shortest path = 35

In dynamic programming we use the below equation to solve the problem

$$g(i,s) = \min_{K \in S} \left\{ C_{ik} + g(k, S - \{k\}) \right\}$$

g = cost of

i = Some starting vertex

s = set of vertices

ik = i to k

Our given problem changed according to the equation of dynamic programming

$$g(1, \{2,3,4\}) = \min_{K \in \{2,3,4\}} \left\{ C_{1k} + g(k, \{2,3,4\} - \{k\}) \right\}$$

$$g(2,\varphi)=5$$

$$g(3,\varphi)=6$$

$$g(4,\varphi)=8$$

$$g(2,\{3\})=15$$

$$g(2,\{4\})=18$$

$$g(3,\{2\})=18$$

$$g(3,\{4\})=20$$

$$g(4,\{2\})=13$$

$$g(4,\{3\})=15$$

$$g(2,\{3,4\})=25$$

$$g(3,\{2,4\})=25$$

$$g(4,\{2,3\})=23$$

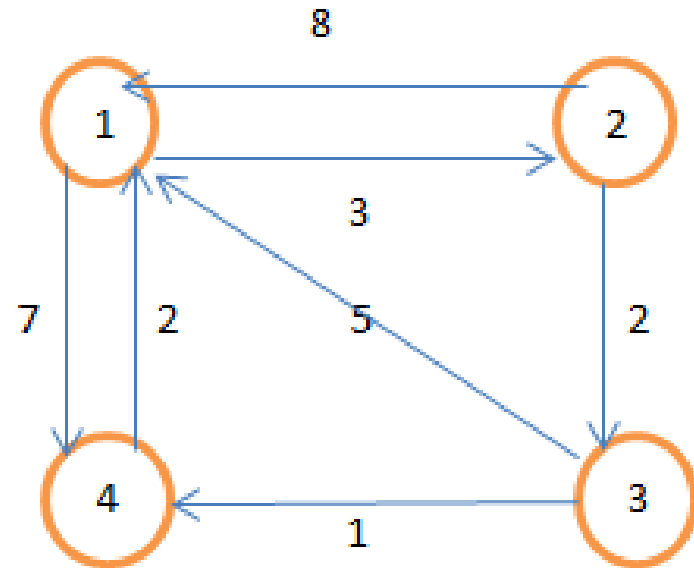
$$\begin{aligned} g(1,\{2,3,4\}) &= \min((C_{12} + g(2,\{3,4\})), C_{13} + g(3,\{2,4\}), C_{14} + g(4,\{2,3\})) \\ &= 10+25, 15+25, 20+23 \\ &= 35 \end{aligned}$$

1----->2----->4-----> 3 = Minimum Value Path

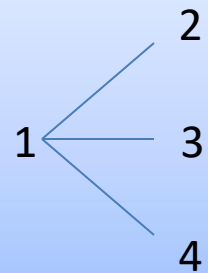
Dynamic Programming
Methodology To Solve The Problem
(Here we have selected vertex 1 as the root node)

ALL PAIR SHORTEST PATH PROBLEM

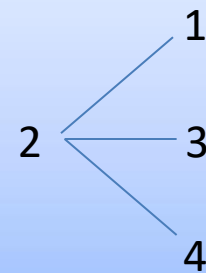
	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0



Starting vertex is '1' then find a path from



Starting vertex is '2' then find a path from



Dijkstra algorithm will find the shortest path from one of the source vertex.

Dynamic programming : A problem should be solved by taking sequence of decisions.
In each stage we will take a decision.

If we have to find a shortest path between vertex 1 $\rightarrow$ 2

There will be a directed path from 1 $\rightarrow$ 2 but there may be a chance of deriving a shortest path going via vertex '3' or it may be going via vertex '4'.

So in each stage we to check or decide a shortest path is going via other vertex.

Absence of edge is taken as ∞ (infinity)

In the case of no self loop we mark as '0'.

$$A^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

- ❑ Now we are going to consider the intermediate vertex as '1' .
- ❑ We check to find whether a better shortest path is present or going via vertex '1'.
- ❑ All path belongs to vertex '1' will remain unchanged.
- ❑ In the step '1' First row and first column has no change while comparing with A^0
- ❑ Below shows the matrix A^1 derived from A^0

$$A^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{bmatrix} \end{matrix}$$

First Check 2 -> 3

$$A^0 \quad [2,3] = 2$$

$$A^0 \quad [2,1] + A^0 \quad [1,3]$$
$$8 \quad + \quad \infty$$

*There is no better path +

First Check 2 -> 4

$$A^0 \quad *2,4+ = \infty$$

$$A^0 \quad [2,1] + A^0 \quad [1,4]$$
$$8 \quad + \quad 7$$

[Since there is less value (15) compared to ∞ we will update the matrix with value 15 +

First Check 3 -> 2

$$A^0 \quad *3,2+ = \infty$$

$$A^0 \quad [3,1] + A^0 \quad [1,2]$$
$$5 \quad + \quad 3$$

[Since there is less value (8) compared to ∞ we will update the matrix with value 8+

First Check 3 -> 4

$$A^0 \quad [3,4] = 1$$

$$A^0 \quad [3,1] + A^0 \quad [1,4]$$
$$5 \quad + \quad 7$$

[Since the value (12) is greater than 1 there will be no change in the matrix]

First Check 4 -> 2

$$A^0 \quad *4,2+ = \infty$$

$$A^0 \quad [4,1] + A^0 \quad [1,2]$$
$$2 \quad + \quad 3$$

[Since there is less value (5) compared to ∞ we will update the matrix with value 5 +

First Check 4 -> 3

$$A^0 * 4,3+ = \infty$$

$$A^0 [4,1] + A^0 [1,3]$$

$$2 +$$

$$\infty$$

*Since the value (2+ ∞) is greater than ∞ there will be no change in the matrix]

Considering '2' as the intermediate vertex

In the step '2' Second row and second column has no change while comparing with A^1

Below shows the matrix A^2 derived from A^1

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

First Check 1 -> 3

$$A^1 [1,3] = \infty$$

$$A^1 [1,2] + A^1 [2,3]$$

$$3 + 2 \quad [\text{Since there is less value (5)}$$

compared to ∞ we will update the matrix with value 5]

First Check 1 -> 4

$$A^1 [1,4] = 7$$

$$A^1 [1,2] + A^1 [2,4]$$

$$3 + 15 \quad [\text{NO UPDATE}]$$

First Check 3 -> 1

$$A^1 [3,1] = 5$$

$$A^1 [3,2] + A^1 [2,1]$$

$$8 + 8 \quad [\text{NO UPDATE}]$$

First Check 3 -> 4

$$A^1 [3,4] = 1$$

$$A^1 [3,2] + A^1 [2,4]$$

$$8 + 15 \quad [\text{NO UPDATE}]$$

First Check 4 -> 1

$$A^1 [4,1] = 2$$

$$A^1 [4,2] + A^1 [2,1]$$

$$5 + 8 \quad [\text{NO UPDATE}]$$

First Check 4 -> 3

$$A^1 [4,3] = \infty$$

$$A^1 [4,2] + A^1 [2,3]$$

$$5 + 2 \quad [\text{UPDATE}]$$

Considering '3' as the intermediate vertex

In the step '3' Third row and third column has no change while comparing with A^2

Below shows the matrix A^3 derived from A^2

$$A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

First Check 1 $\rightarrow$ 2

$$\begin{array}{l} A^2 \quad [1,2] = 3 \\ A^2 \quad [1,3] + A^2 \quad [3,2] \\ 5 \quad + \quad 8 \quad [\text{NO UPDATE}] \end{array}$$

First Check 1 $\rightarrow$ 4

$$\begin{array}{l} A^2 \quad [1,4] = 7 \\ A^2 \quad [1,3] + A^2 \quad [3,4] \\ 5 \quad + \quad 1 \quad [\text{UPDATE}] \end{array}$$

First Check 2 -> 1

$$A^2 \quad [2,1] = 8$$

$$A^2 \quad [2,3] + A^2 \quad [3,1]$$

$$2 \quad + \quad 5 \quad [\text{UPDATE}]$$

First Check 2 -> 4

$$A^2 \quad [2,4] = 15$$

$$A^2 \quad [2,3] + A^2 \quad [3,4]$$

$$2 \quad + \quad 1 \quad [\text{UPDATE}]$$

First Check 4 -> 1

$$A^2 \quad [4,1] = 2$$

$$A^2 \quad [4,3] + A^2 \quad [3,1]$$

$$7 \quad + \quad 5 \quad [\text{NO UPDATE}]$$

First Check 4 -> 2

$$A^2 \quad [4,2] = 5$$

$$A^2 \quad [4,3] + A^2 \quad [3,2]$$

$$7 \quad + \quad 8 \quad [\text{NO UPDATE}]$$

Considering '4' as the intermediate vertex

In the step '4' Fourth row and fourth column has no change while comparing with A^3

Below shows the matrix A^4 derived from A^3

$$A^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

First Check 1 -> 2

$$\begin{array}{l} A^3 \quad [1,2] = 3 \\ A^3 \quad [1,4] + A^3 \quad [4,2] \\ 6 \quad + \quad 5 \quad [\text{NO UPDATE}] \end{array}$$

First Check 1 -> 3

$$\begin{array}{l} A^3 \quad [1,3] = 5 \\ A^3 \quad [1,4] + A^3 \quad [4,3] \\ 6 \quad + \quad 7 \quad [\text{NO UPDATE}] \end{array}$$

First Check 2 -> 1

$$\begin{array}{l} A^3 \quad [2,1] = 7 \\ A^3 \quad [2,4] + A^3 \quad [4,1] \\ \quad 3 \quad + \quad \quad 2 \quad [\text{UPDATE}] \end{array}$$

First Check 2 -> 3

$$\begin{array}{l} A^3 \quad [2,3] = 2 \\ A^3 \quad [2,4] + A^3 \quad [4,3] \\ \quad 3 \quad + \quad \quad 7 \quad [\text{NO UPDATE}] \end{array}$$

First Check 3 -> 1

$$\begin{array}{l} A^3 \quad [3,1] = 5 \\ A^3 \quad [3,4] + A^3 \quad [4,1] \\ \quad 1 \quad + \quad \quad 2 \quad [\text{UPDATE}] \end{array}$$

First Check 3 -> 2

$$\begin{array}{l} A^3 \quad [3,2] = 8 \\ A^3 \quad [3,4] + A^3 \quad [4,2] \\ \quad 1 \quad + \quad \quad 5 \quad [\text{UPDATE}] \end{array}$$

Generally :

$$A^K[i,j] = \min \{A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j]\}$$

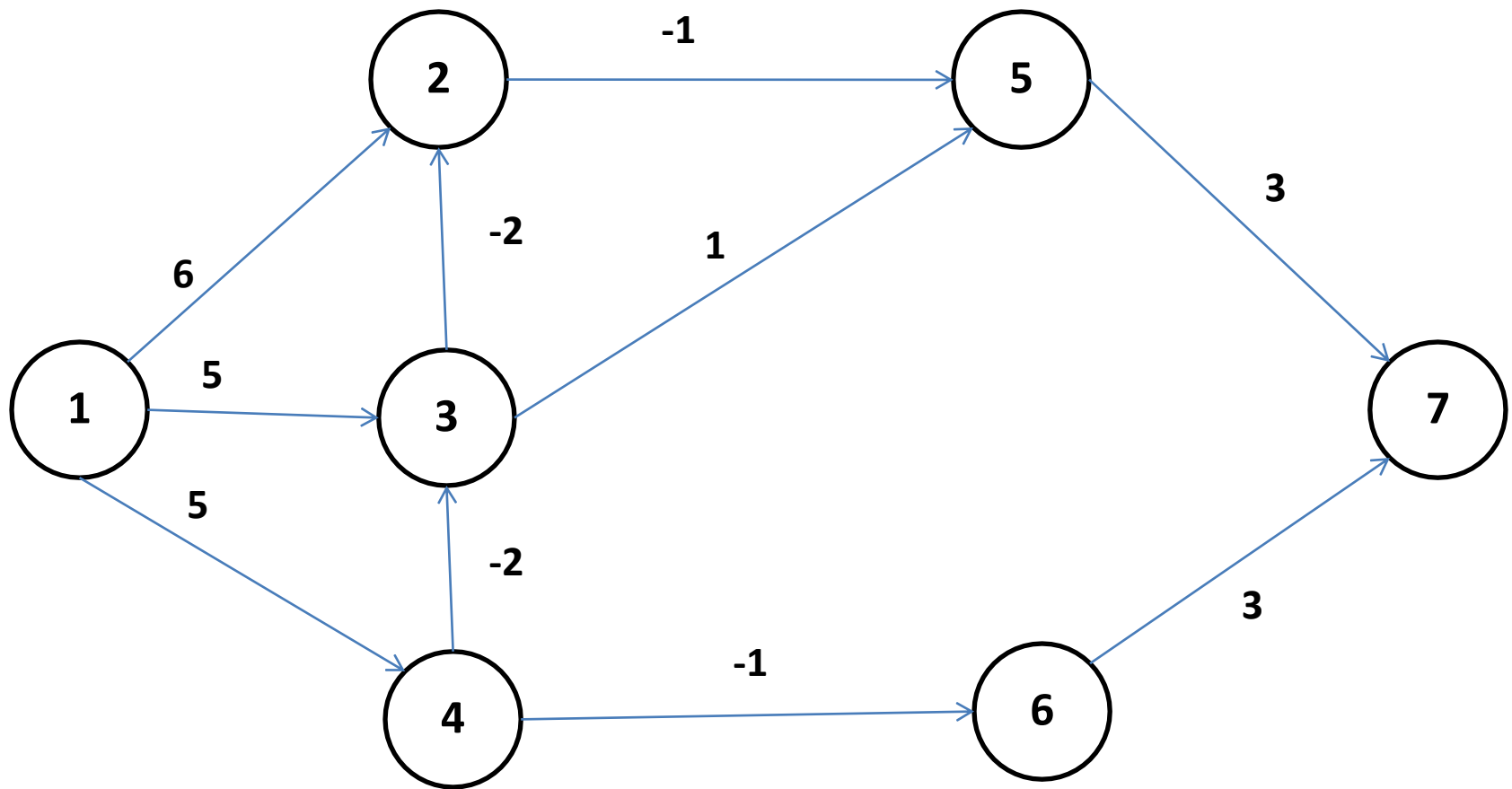
For getting $A[i,j]$ of any intermediate vertex 'k' we are taking the minimum of $A[i,j]$ from which the previous matrix 'k-1' either this or $A^{k-1}[i,k] + A^{k-1}[k,j]$. 'k' is introduced in between and addition is done

Programming Code :

```
for(k=1;k<=n;k++)
{
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            A[i,j] = min(A[i,j],A[i,k]+A[k,j]);
        }
    }
}
```

Time = $O(n^3)$

Bellman Ford Algorithm

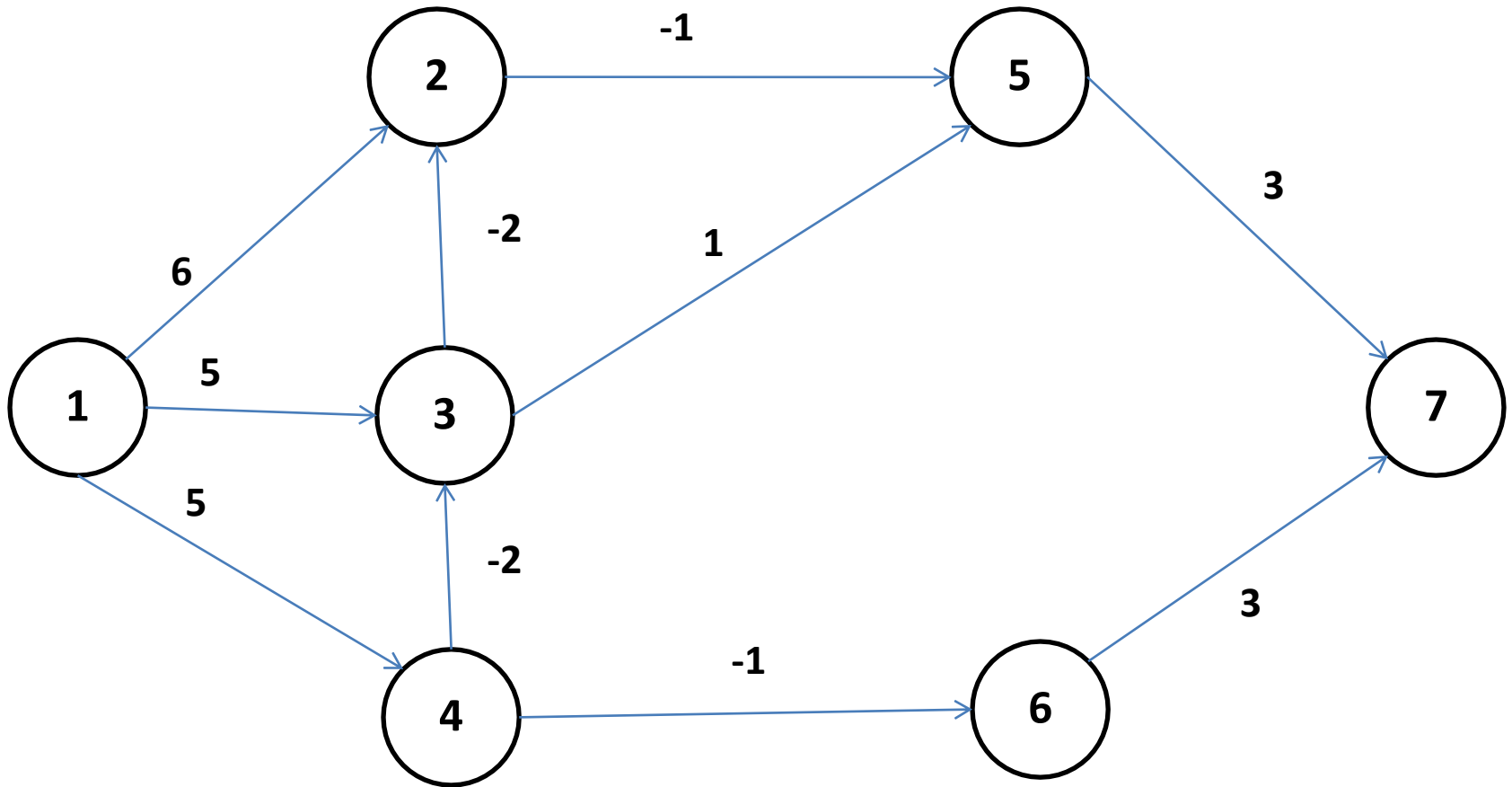


❑ Take one of the vertex as the source vertex & find out all other paths. Dijkstra Algorithm will not work if there are negative edge(It may or may not work correctly).

❑ In Bellman Ford we have to try out all possible solution & pick up the best solution.

❑ Go on relaxing all the edges

❑ Relaxation is the most important step in Bellman-Ford. It is what increases the accuracy of the distance to any given vertex. Relaxation works by continuously shortening the calculated distance between vertices comparing that distance with other known distances.



Here we will have to relax the edges for $n-1$ times.
' n ' is the number of vertices

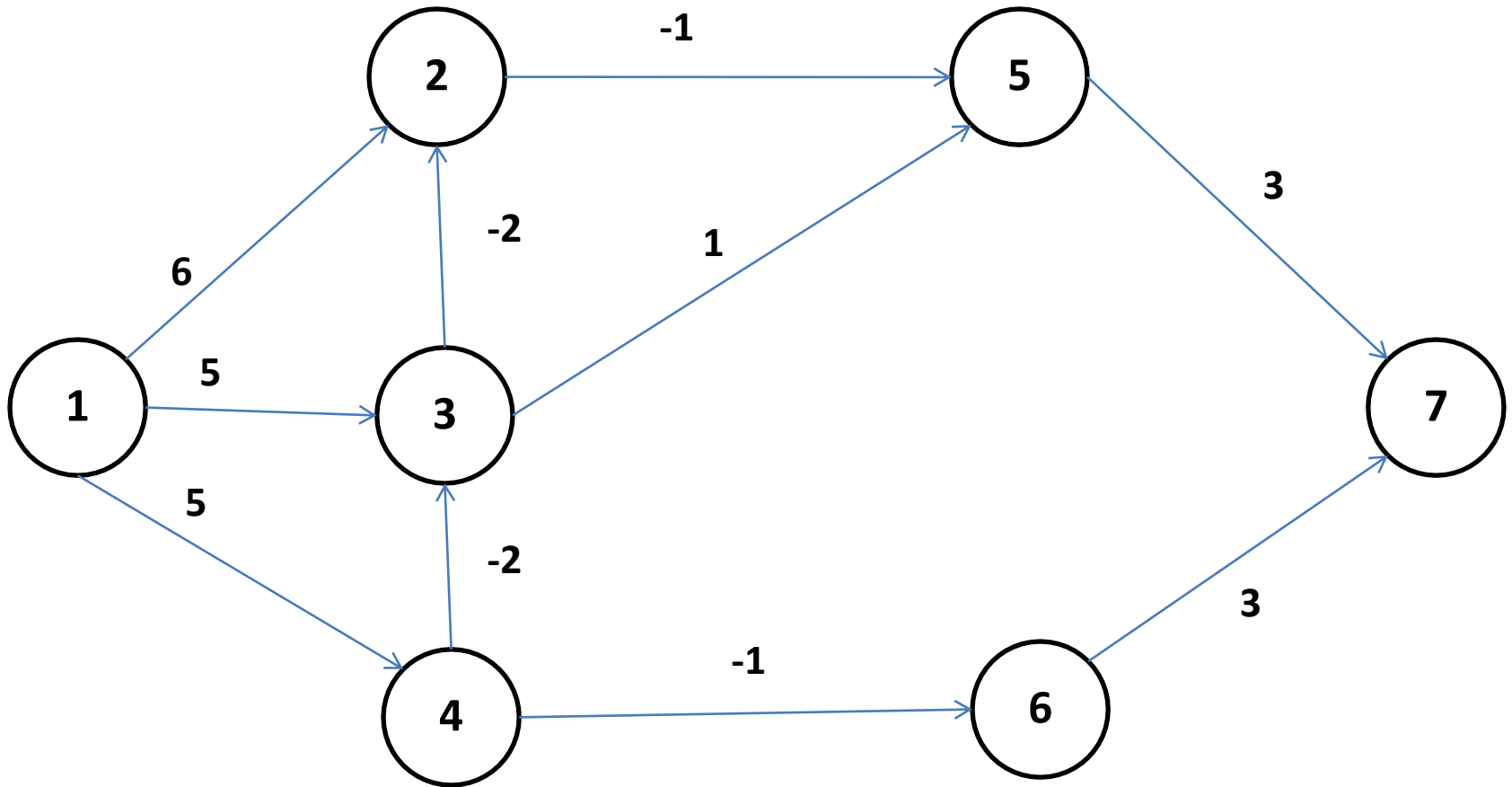
Relaxation :

Suppose there is an edge (u,v)

If ($d[u] + c(u,v) < d[v]$)

$d[v] = d[u] + c(u,v)$

‘u’ is the first vertex ‘v’ is the second vertex. If there is an edge (u,v) then check the distance of ‘u’ + cost of an edge (u,v) is less than distance of v if so then change the distance of vertex ‘v’ to distance of ‘u’ + cost of an edge (u,v)



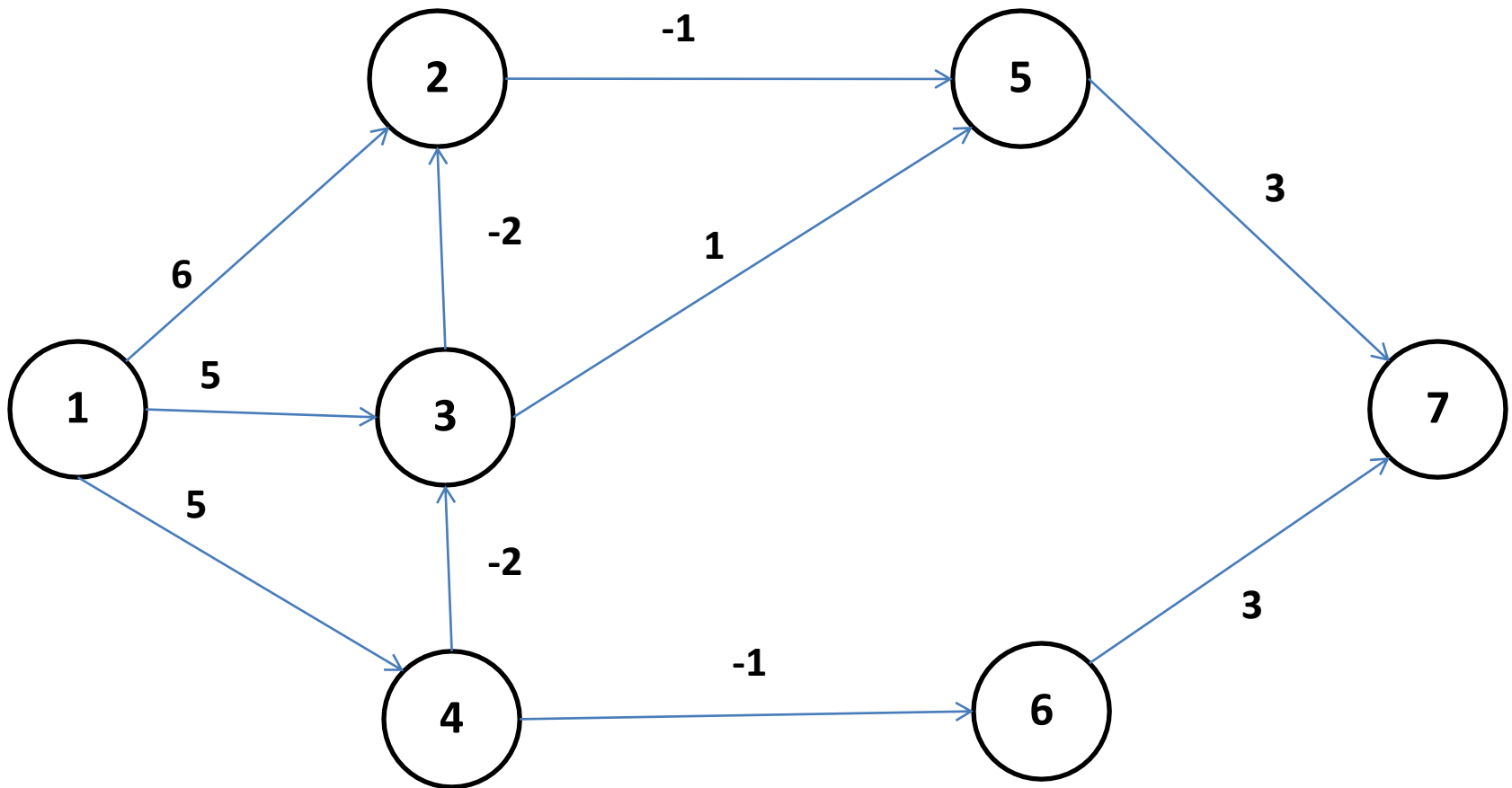
Prepare the list of edges (Here total 10 edges)

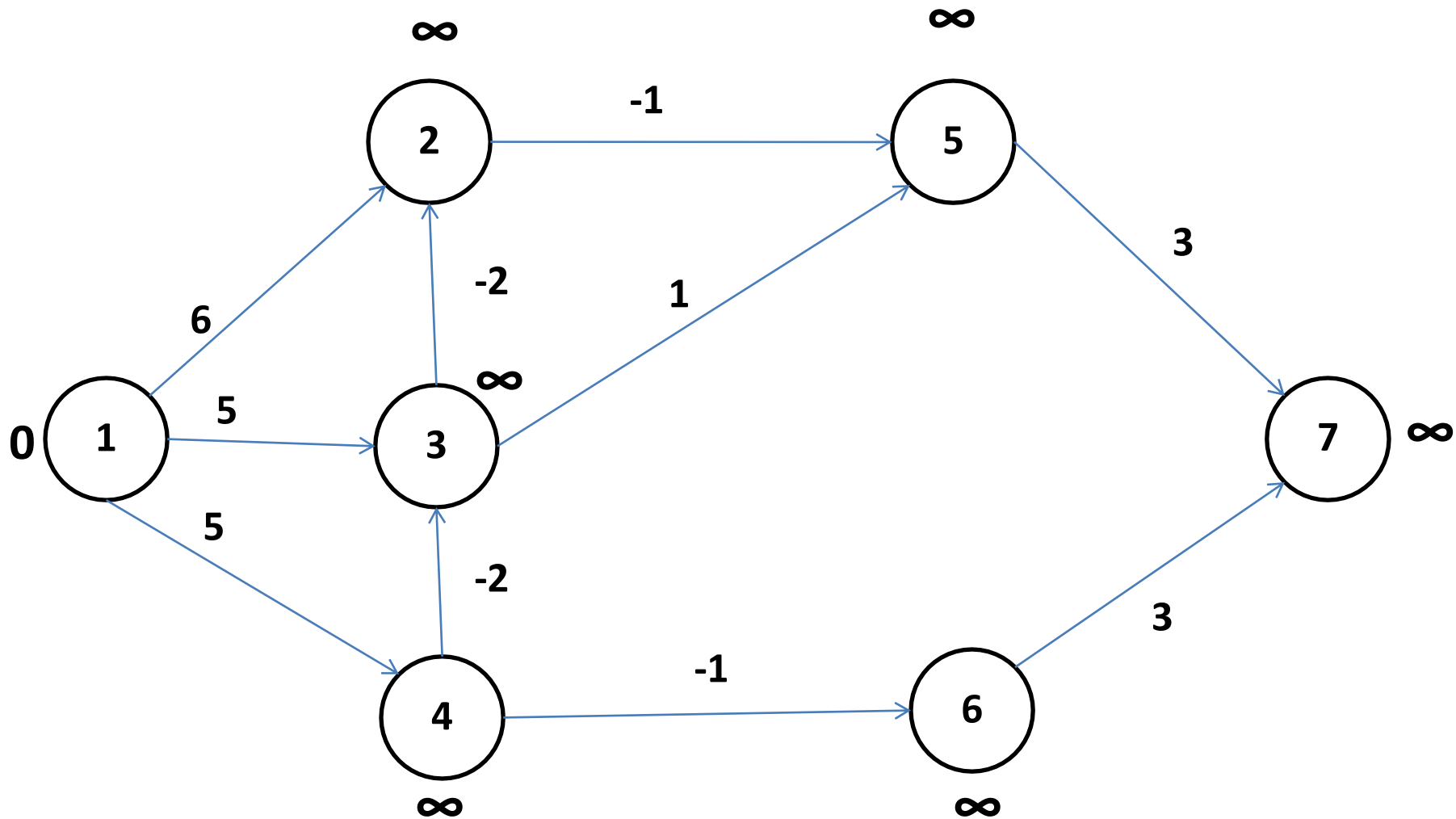
Edge list = (1,2), (1,3), (1,4), (2,5), (3,2), (3,5), (4,3), (4,6), (5,7), (6,7)

Now we have to relax all these edges for 6 times.

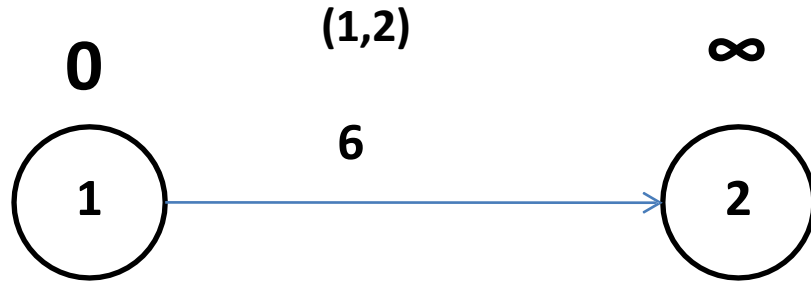
Initially we have to mark the distance of all these vertices.

Put the distance of starting vertex as '0' and mark all other vertex distance as ' ∞ '

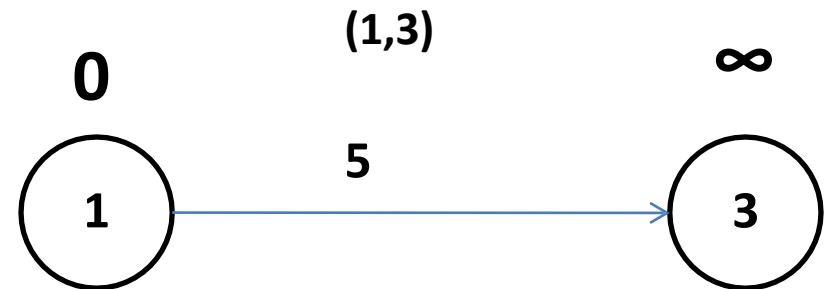
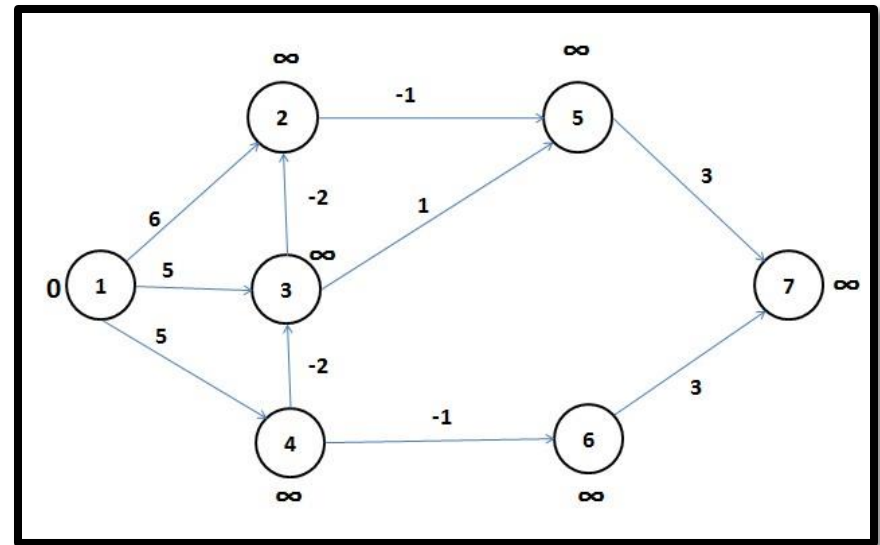
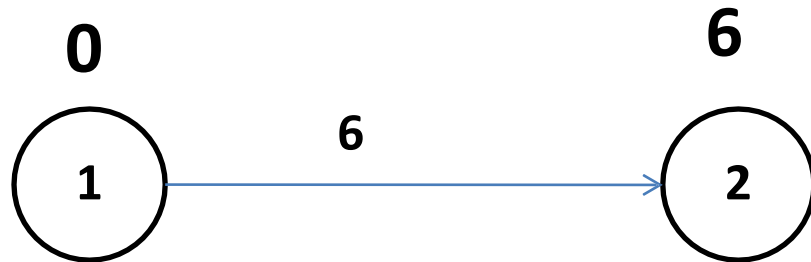




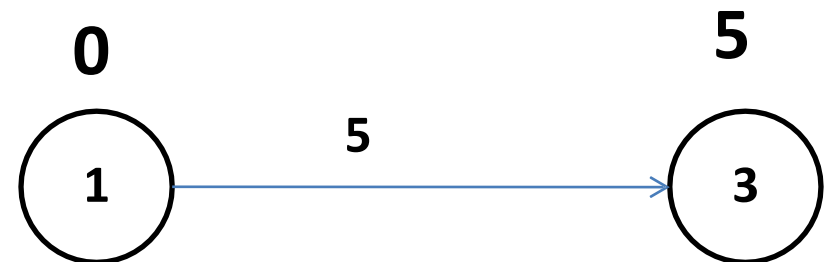
Relaxation for the first time

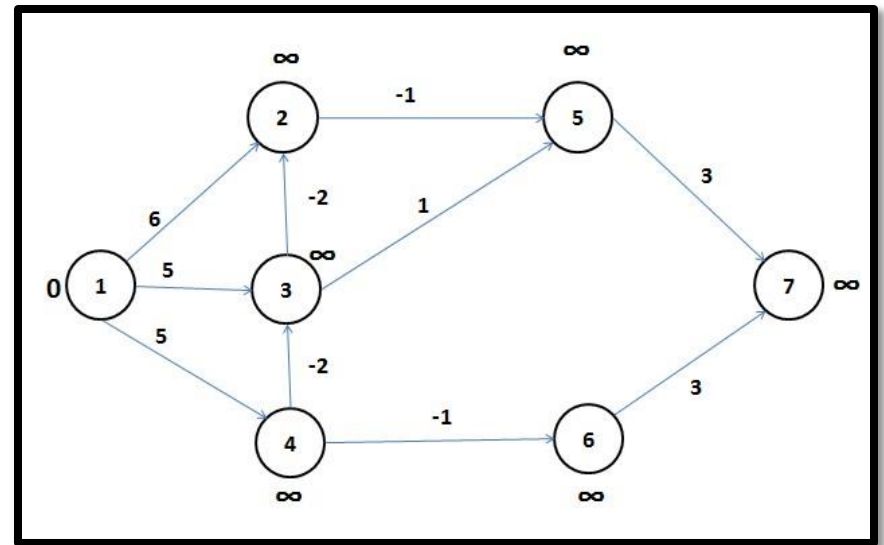
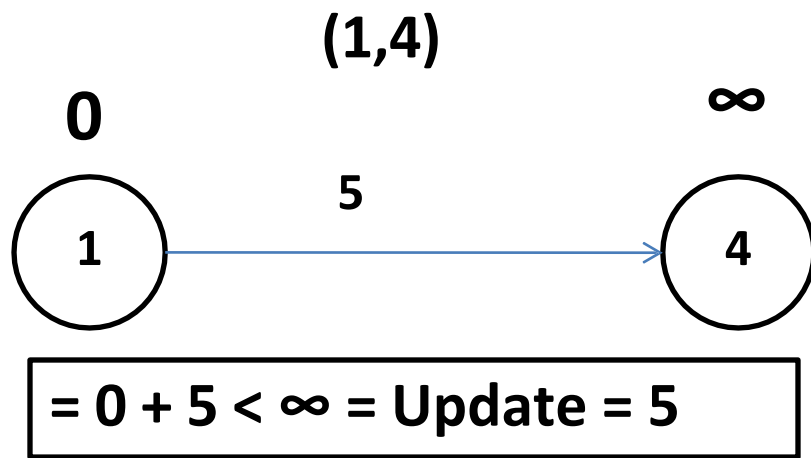


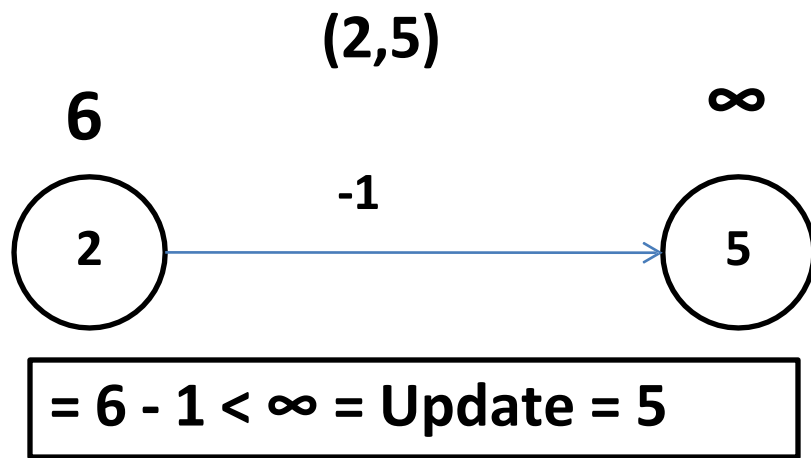
$$= 0 + 6 < \infty = \text{Update} = 6$$

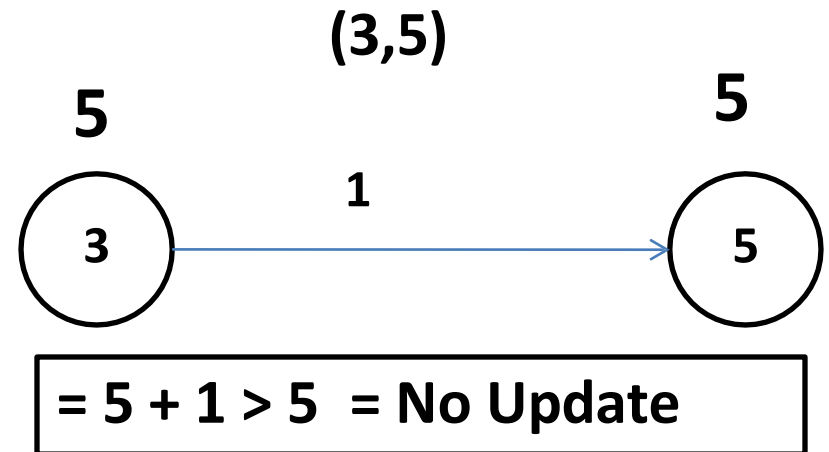
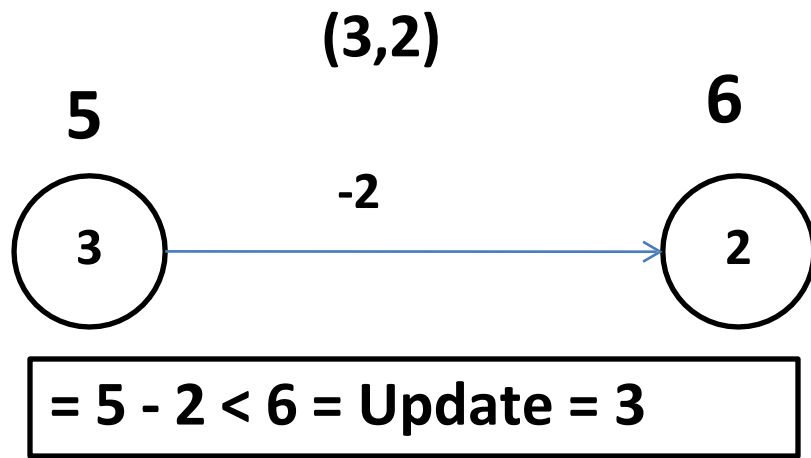


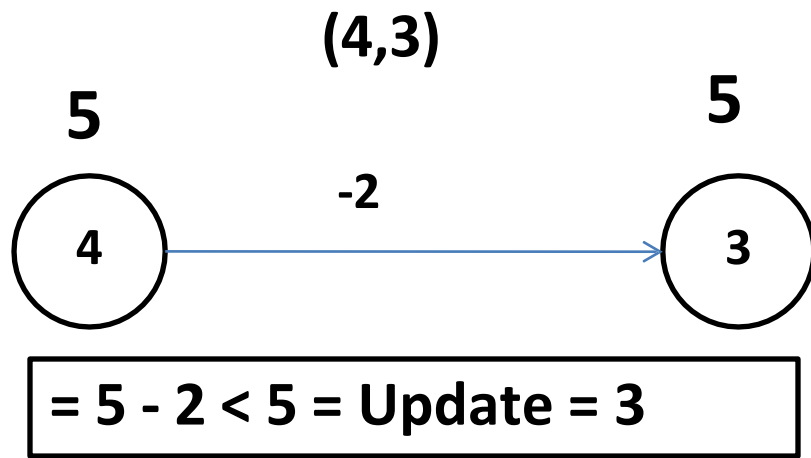
$$= 0 + 5 < \infty = \text{Update} = 5$$

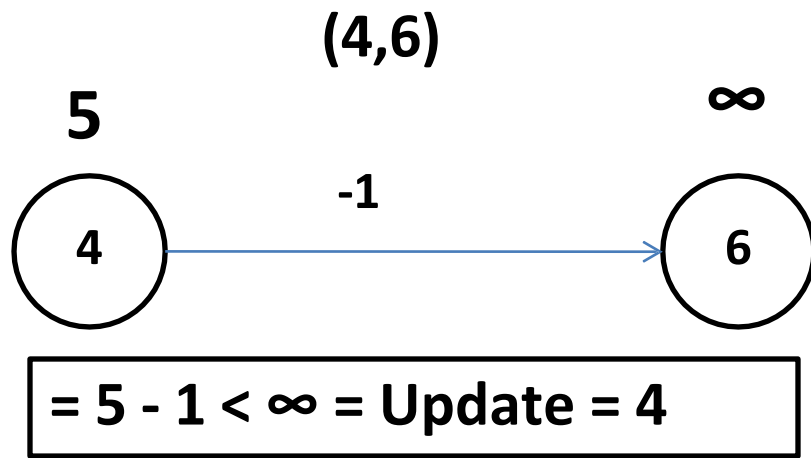


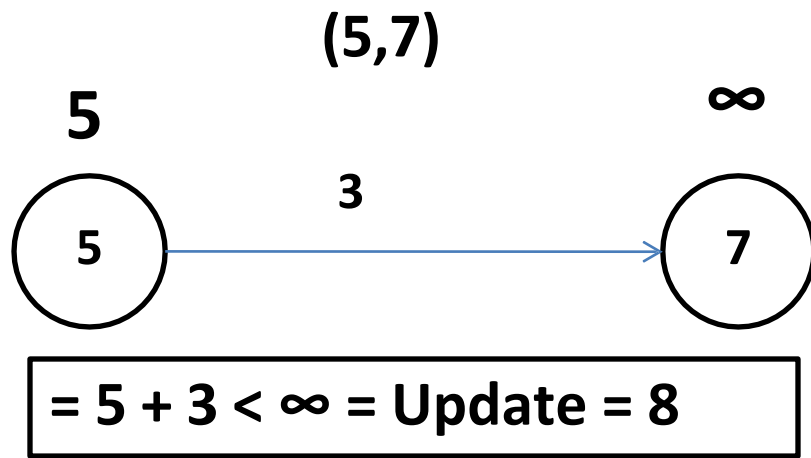


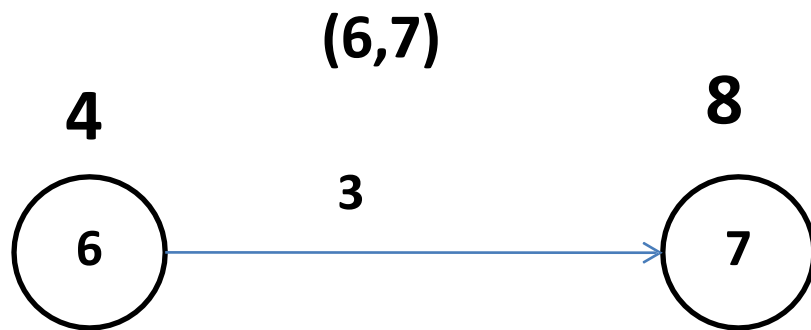




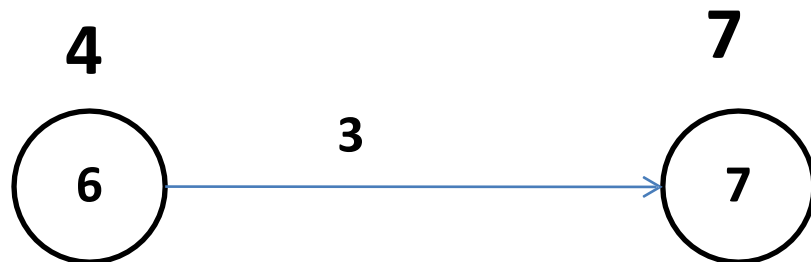


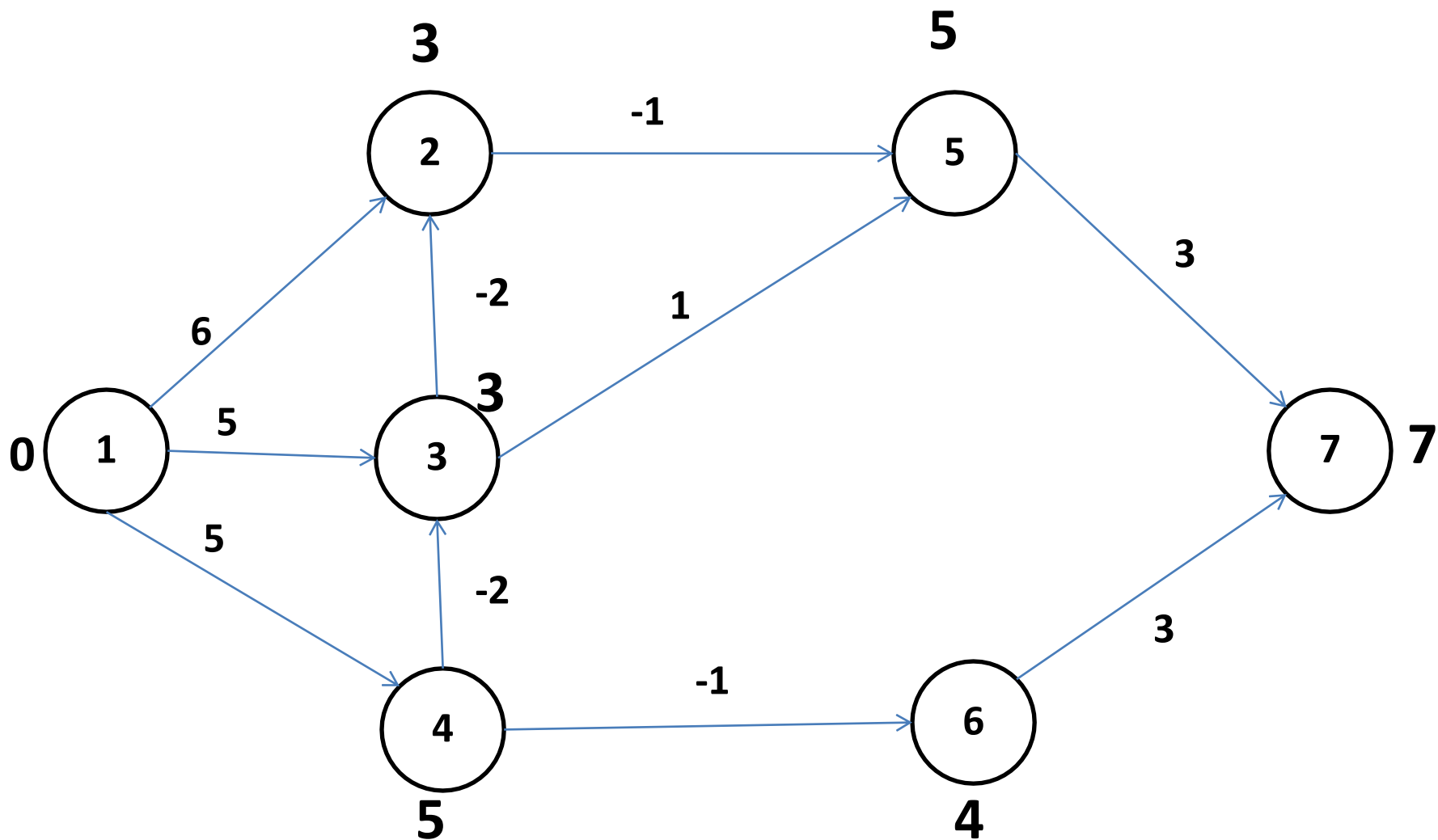




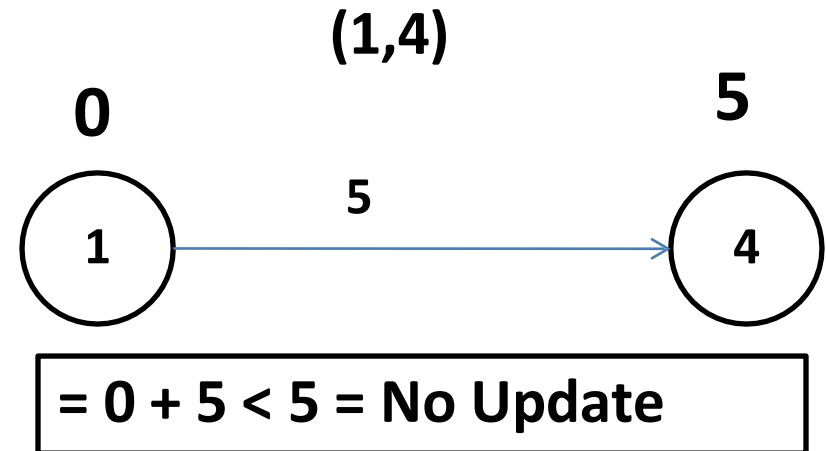
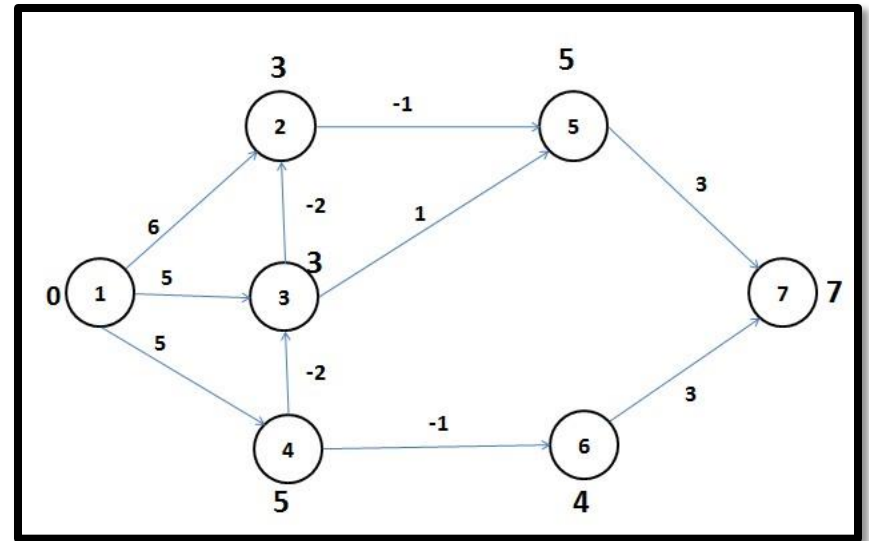
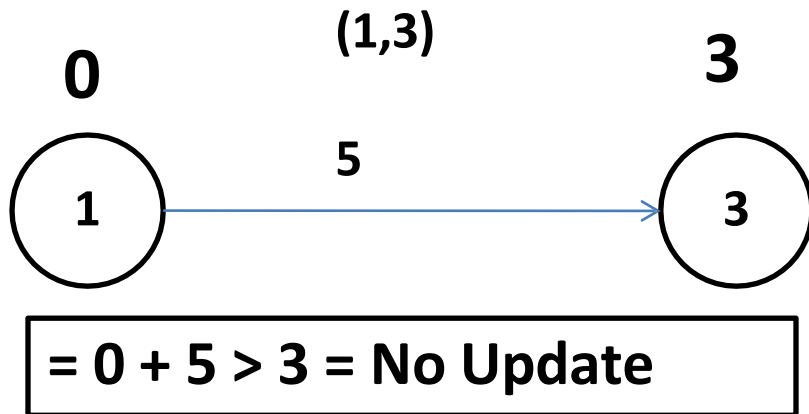
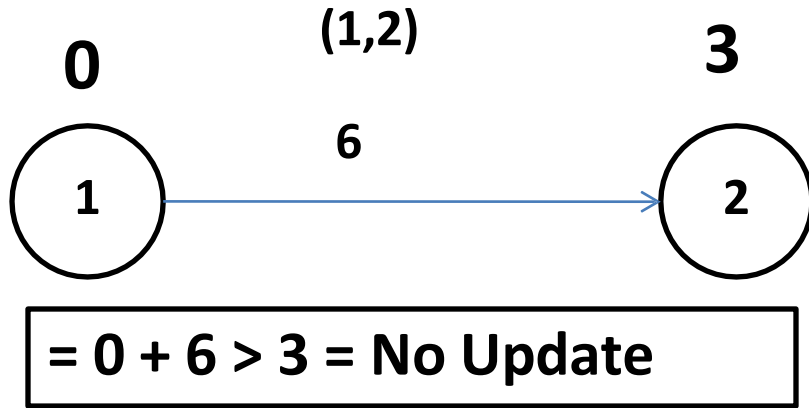


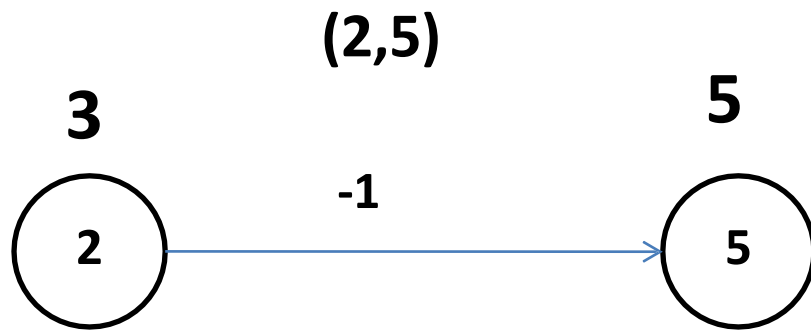
$$= 4 + 3 < 8 = \text{Update} = 7$$



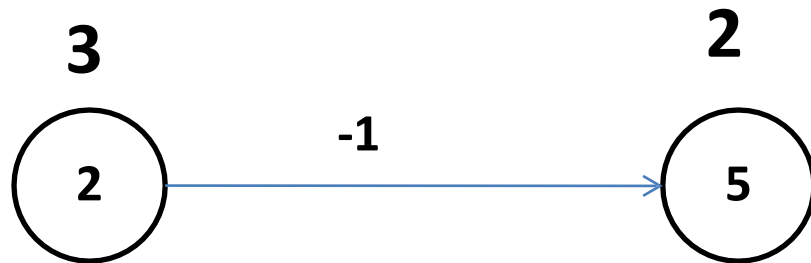


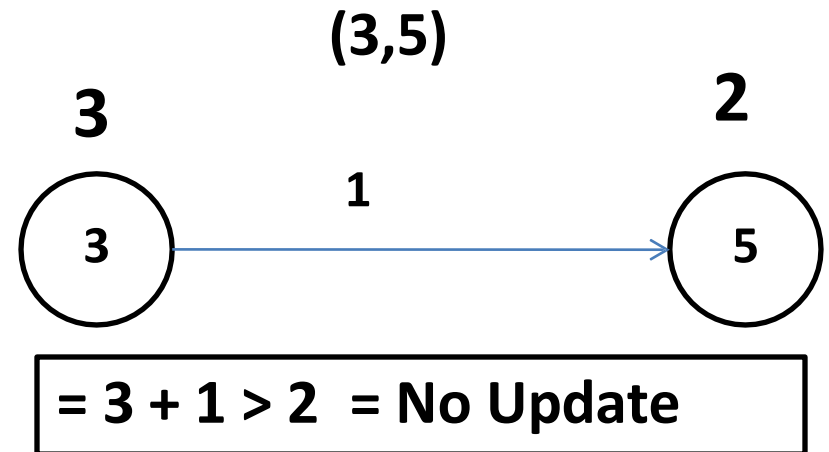
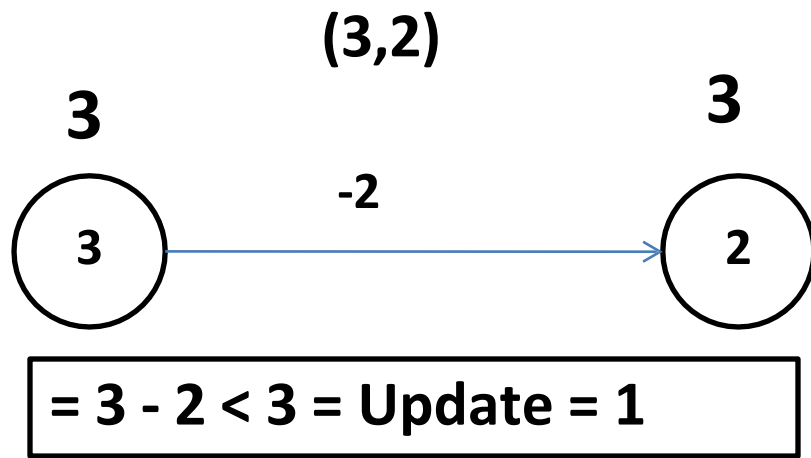
Relaxation for the second time

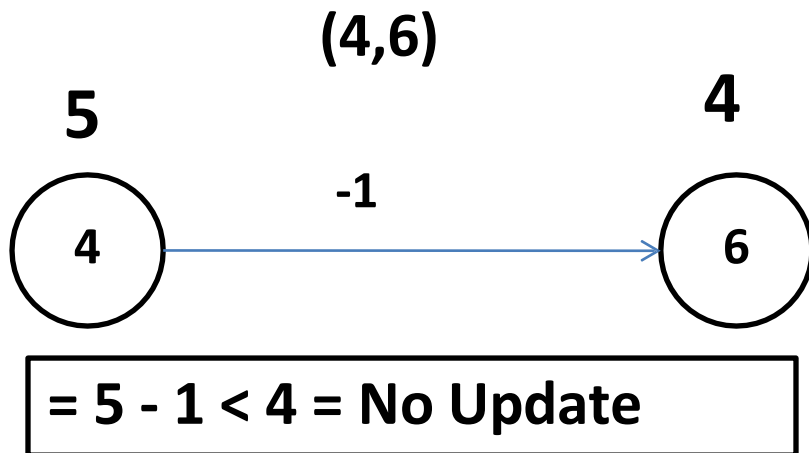
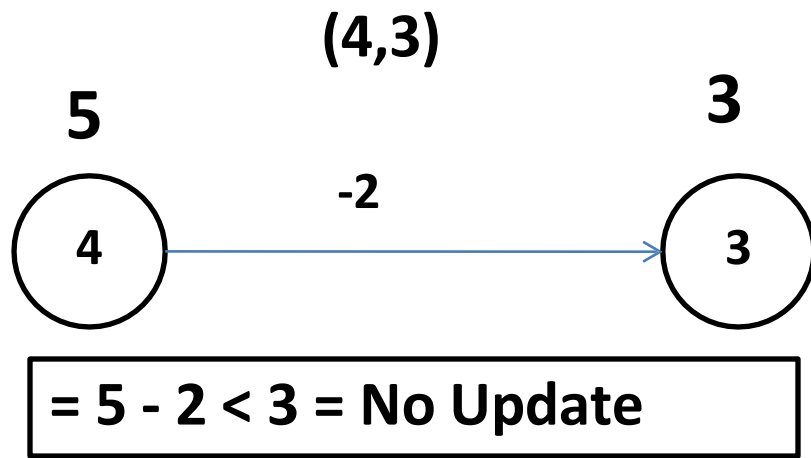


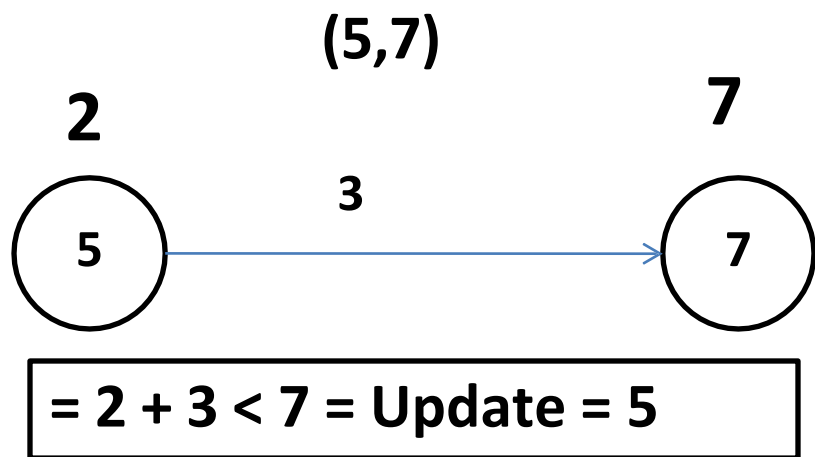


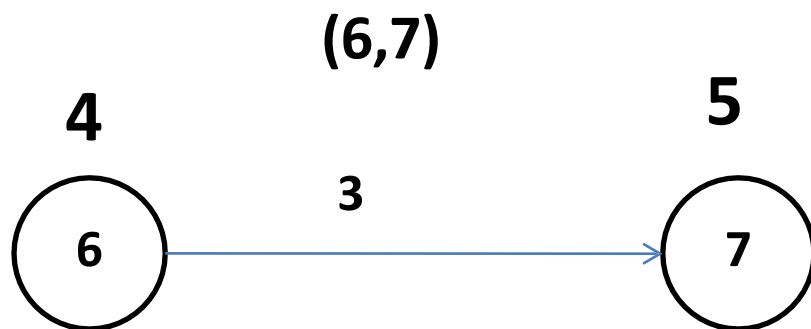
$$= 3 - 1 < 5 = \text{Update} = 2$$



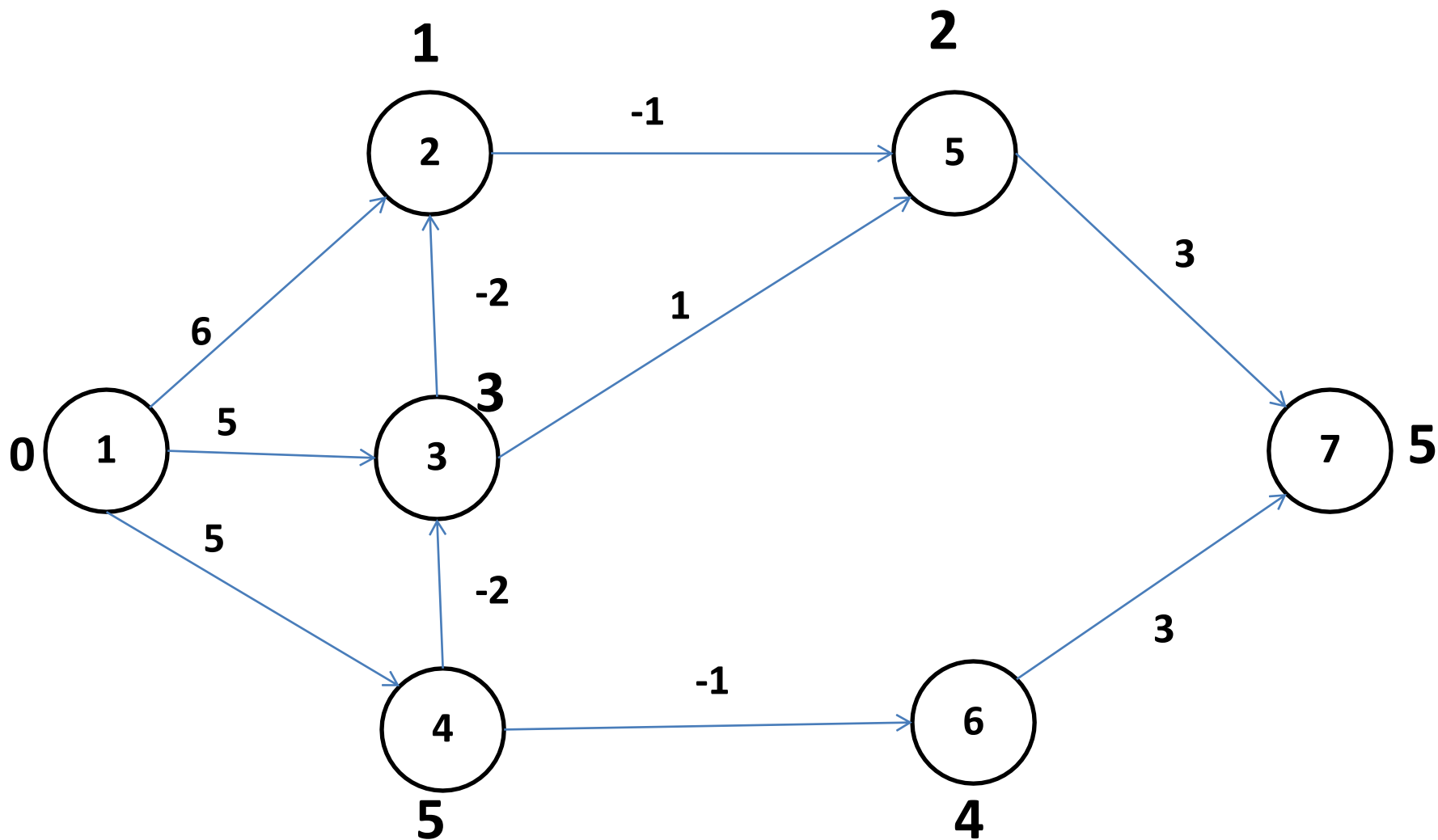




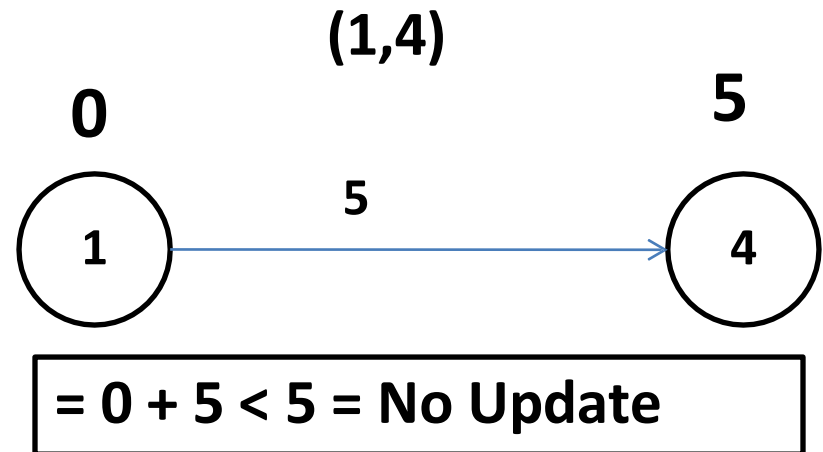
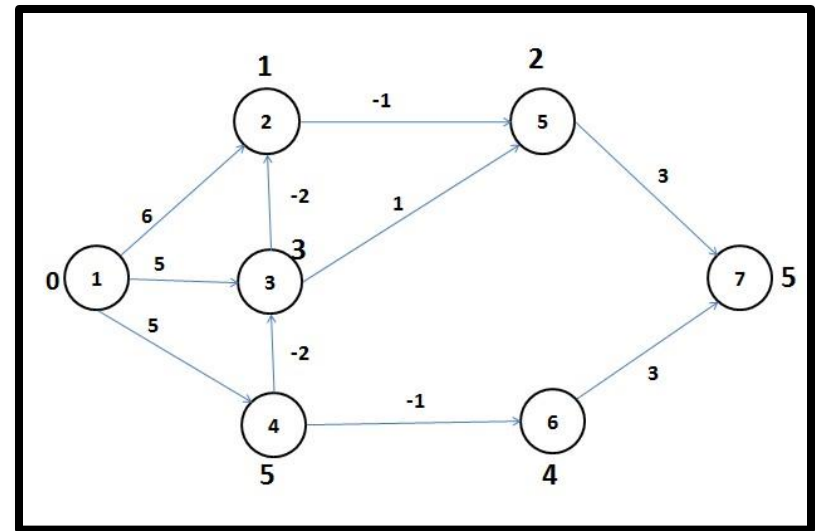
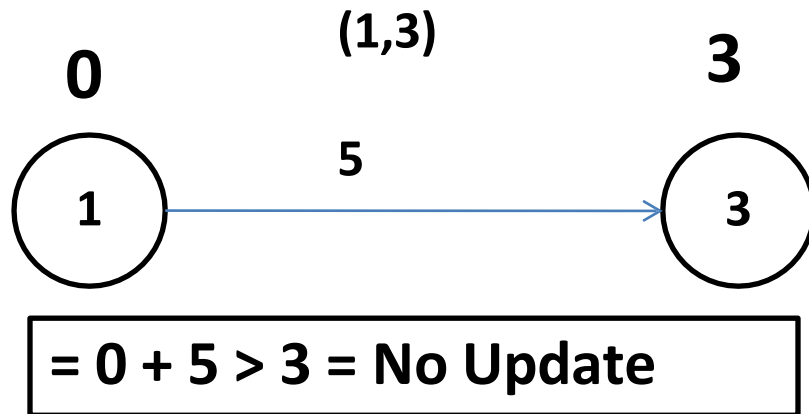
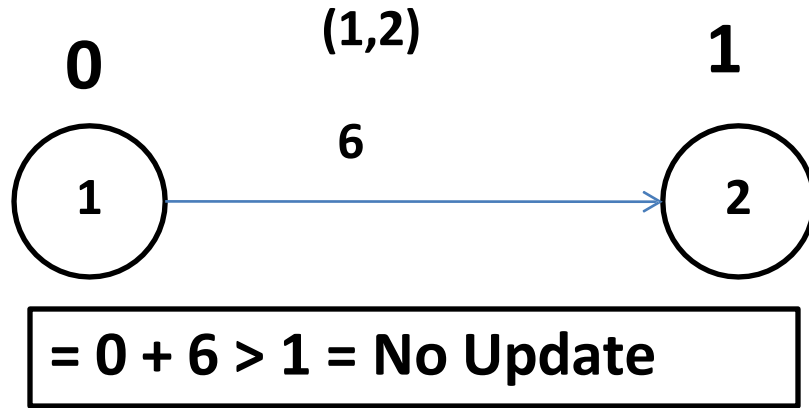


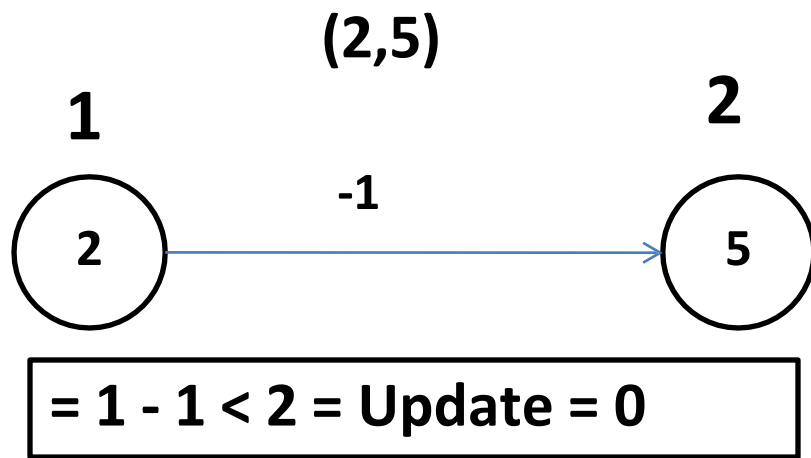


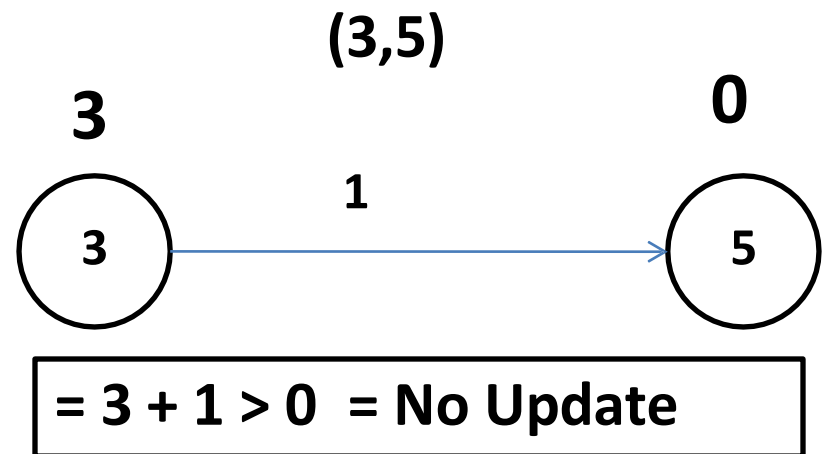
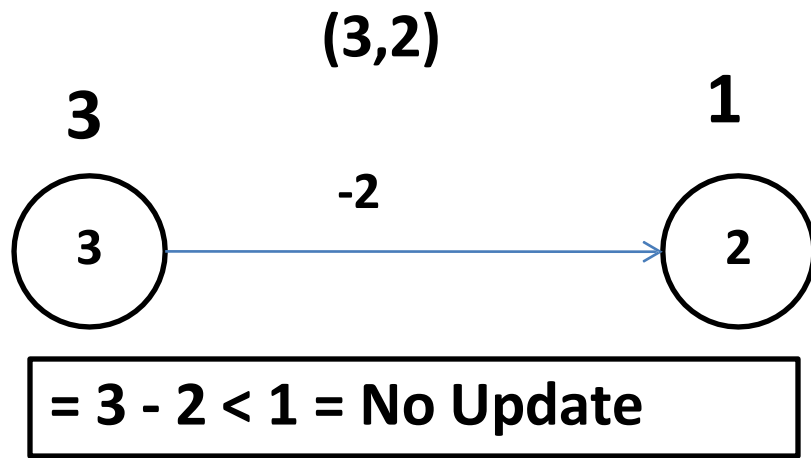
$= 4 + 3 > 5 = \text{No Update}$

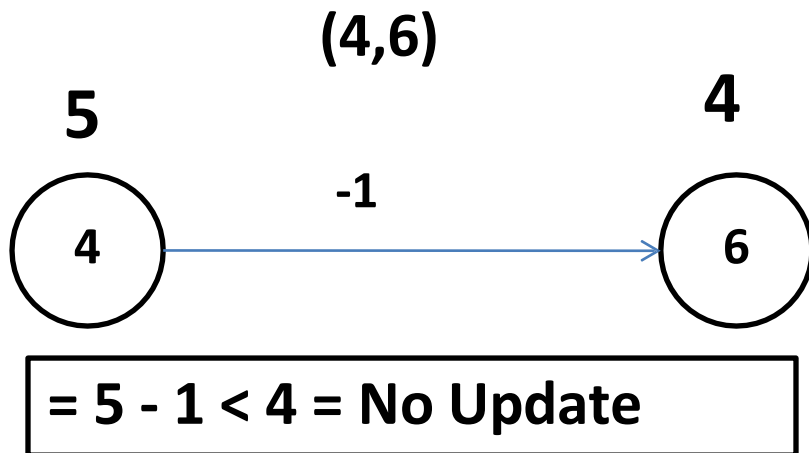
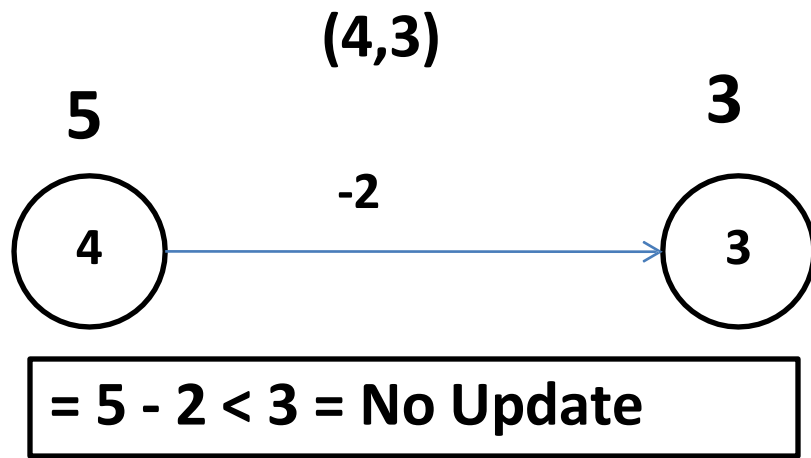


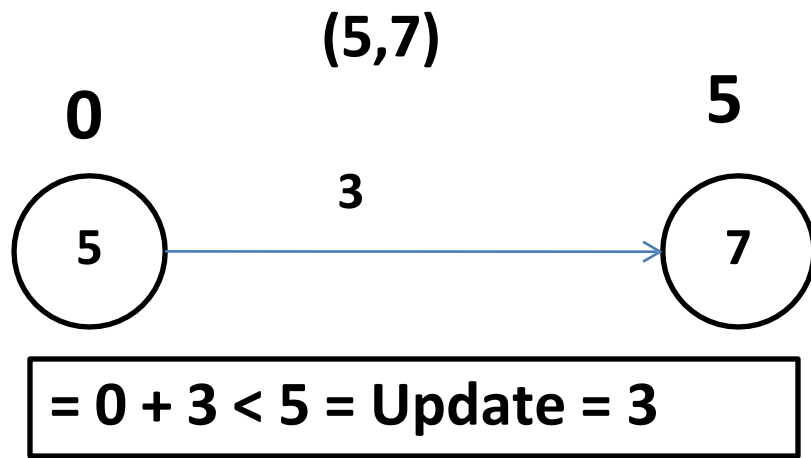
Relaxation for the third time

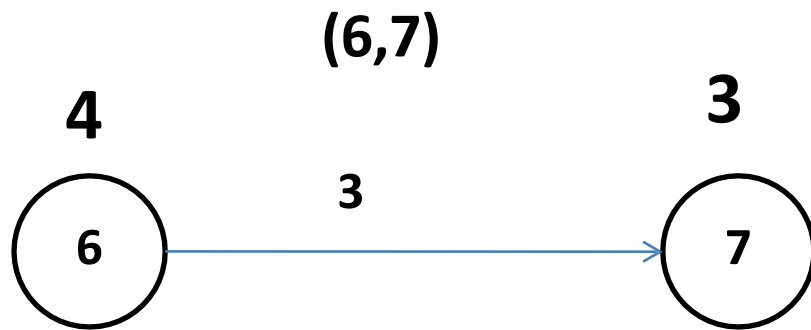




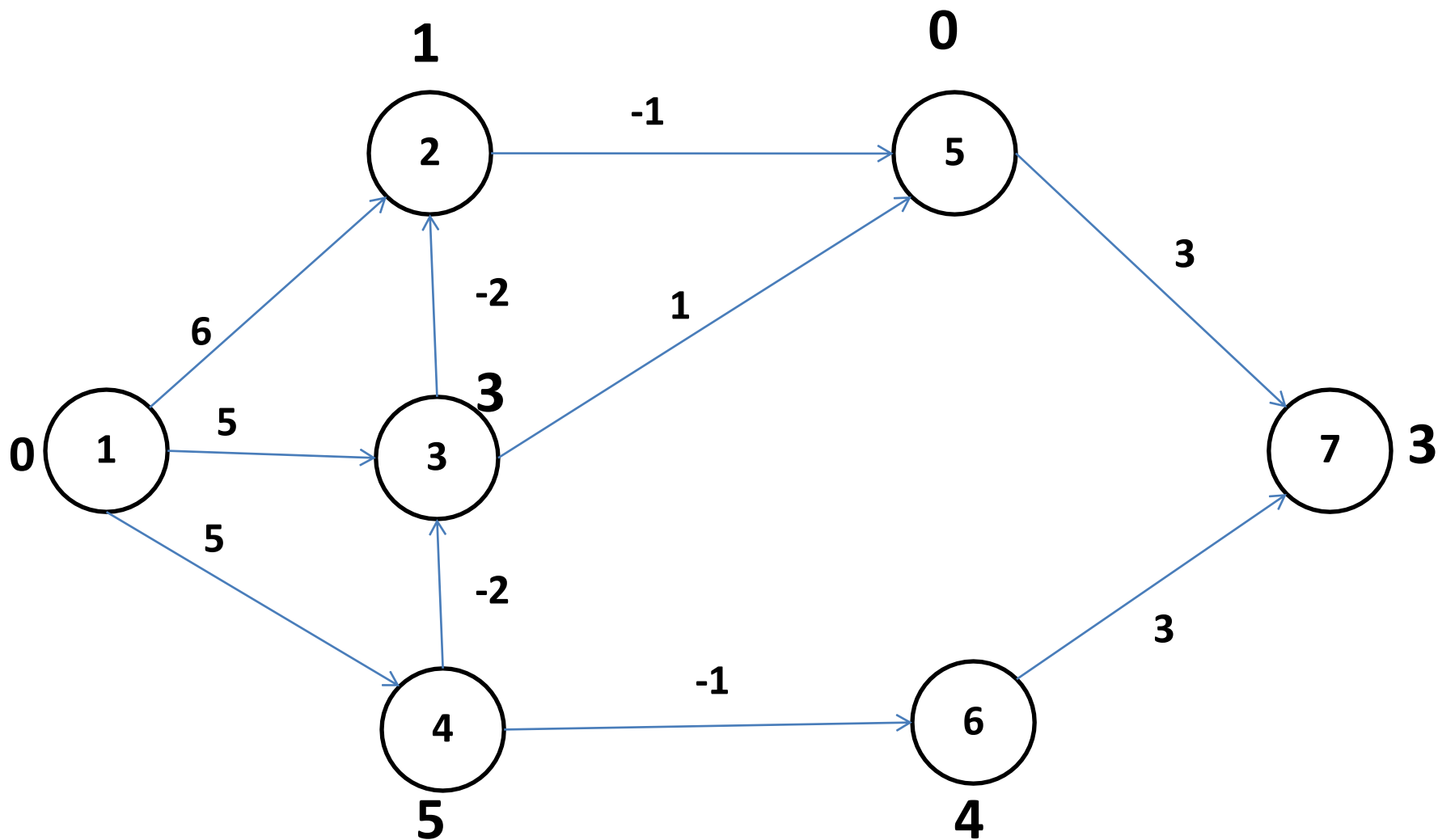








$= 4 + 3 > 3 = \text{No Update}$



Vertex	Distance
1	0
2	1
3	3
4	5
5	0
6	4
7	3